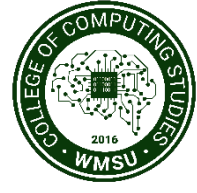




Republic of the Philippines
Western Mindanao State University
College of Computing Studies
DEPARTMENT OF COMPUTER SCIENCE
Zamboanga City



BalotaChain on the Saksi Framework: End-to-End Verifiable Elections on Permissioned Blockchain Using Synthetic Election Data

A Thesis Presented to the Faculty of

Department of Computer Science

College of Computing Studies

In Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Computer Science

VANIEL JOHN G. CORNELIO

FREDERICK F. BEJERANO

IDRIS S. BAKAUN

Researchers

MARK L. FLORES, PhD

Adviser

June 2026

Table of Contents

Table of Contents.....	ii
List of Figures.....	iv
List of Tables.....	v
List of Appendices.....	vi
CHAPTER I INTRODUCTION.....	7
Background of the Study.....	7
Statement of the Problem.....	8
Objectives.....	9
Scope and Limitations.....	10
Significance of the Study.....	11
Definition of Terms.....	11
CHAPTER II REVIEW OF RELATED LITERATURE.....	16
2.1 Introduction.....	16
2.2 Cryptographic Foundations of Verifiable Voting.....	17
2.3 End-to-End Verifiable Voting Systems.....	19
2.4 Blockchain-Based Voting Systems.....	20
2.5 Security Models and Verifiability Definitions.....	21
2.6 Electronic Voting in Practice.....	22
2.7 Synthesis and Research Gap.....	24
2.8 Review Matrix.....	26
2.9 Conceptual Framework.....	28
CHAPTER III METHODOLOGY.....	30

Research Design.....	30
The Saksi Framework.....	34
Data Collection.....	37
Algorithms.....	39
Algorithm 1: Threshold ElGamal Encryption on ristretto255.....	40
Algorithm 2: Pedersen Distributed Key Generation.....	40
Algorithm 3: Zero-Knowledge Proofs.....	40
Algorithm 4: Blind-Signed Anonymous Credentials.....	41
Software Development.....	42
Software Tools.....	44
Testing and Evaluation.....	45
Experimental Design.....	47
Accuracy (demonstrating integrity).....	50
Privacy.....	51
Threat Simulation and Mapping to Philippine Electoral Law.....	53
Election Return Verification and Approval.....	55
Audit Trails and Logging.....	55
Performance.....	57
Ethical Considerations.....	59
References.....	60

List of Figures

<i>Figure 2.1.</i> Layered composition of cryptographic primitives in a verifiable voting system.....	18
<i>Figure 2.2.</i> Timeline of key developments in cryptographic and blockchain-based voting (1985–2026).....	19
<i>Figure 2.3.</i> Feature-coverage matrix of representative systems against the requirements identified in this review. Among the systems reviewed, only the proposed framework satisfies all seven properties.....	26
<i>Figure 2.4.</i> Conceptual framework of the study: inputs, the Saksi and BalotaChain process chain, measured outputs, and the design and evaluation cycle.....	29
<i>Figure 3.1.</i> Overview of the research methodology following the DSRM process model.....	32
<i>Figure 3.2.</i> Input-Process-Output framework of the study.....	34
<i>Figure 3.3.</i> Workflow of the Saksi framework as instantiated in the BalotaChain election application, from key generation to independent verification.....	36
<i>Figure 3.4.</i> System architecture of BalotaChain with component and trust boundaries.....	43

List of Tables

<i>Table 2.1.</i> Comparison of representative voting systems across key architectural dimensions.....	25
<i>Table 2.2.</i> Review matrix of related studies.....	26
<i>Table 3.1.</i> Application of the DSRM process model to this study.....	33
<i>Table 3.2.</i> Structure of a synthetic voter record (data collection matrix).....	38
<i>Table 3.3.</i> System components, features, and purpose.....	43
<i>Table 3.4.</i> Software tools and estimated cost.....	44
<i>Table 3.5.</i> Evaluation matrix across the two axes.....	45
<i>Table 3.6.</i> Alignment of research questions, objectives, system functions, scenarios, metrics, and evidence.....	46
<i>Table 3.7.</i> Experiment definitions for the evaluation.....	48
<i>Table 3.8.</i> Testing scenarios for the evaluation.....	48
<i>Table 3.9.</i> Threat model matrix of adversary classes, capabilities, attacks, defenses, and expected evidence.....	53
<i>Table 3.10.</i> Mapping of simulated adversary classes to Philippine electoral and data privacy law.....	54
<i>Table 3.11.</i> Data classification of stored items in the audit trail.....	56
<i>Table 3.12.</i> Benchmarking environment specification.....	58

List of Appendices

Appendix A: Data Collection.....	62
Appendix B: Software Design/Prototype.....	64
Appendix C: Evaluation Tool.....	69
Appendix D: Research Budget.....	72

CHAPTER I

INTRODUCTION

Background of the Study

The conduct of trustworthy elections rests on two properties that are difficult to satisfy at the same time: integrity, meaning that every vote is counted exactly as it was cast, and privacy, meaning that no party can link a voter to the contents of their ballot. Traditional paper-based systems and conventional electronic voting machines pursue these properties through procedural safeguards and institutional trust, depending on observers, custody procedures, and the assumed honesty of election authorities. These mechanisms place the burden of trust on the people and institutions administering the election rather than on guarantees that can be independently verified.

Over the past several decades, cryptographic research has sought to replace this procedural trust with mathematical guarantees. End-to-end verifiable voting systems allow a voter to confirm that their vote was recorded as cast and counted as recorded, and allow any observer to confirm that the announced result is correct, without revealing how anyone voted. More recently, blockchain technology has been applied to provide a tamper-evident and decentralized record of cast ballots, addressing the concern that a single authority controls the record of votes. Permissioned blockchains, which restrict participation to known and authorized parties, combine this tamper-evidence with governance and access control over who may participate in maintaining the record.

In the Philippine context, public confidence in the integrity of elections is a recurring concern, and the country has a documented history of debate over the transparency and auditability of its automated election systems [30], [31]. While cryptographic and blockchain-based voting have been studied extensively in the international literature, there is only limited and design-specific empirical

evidence describing how such a Blockchain by itself provides an append-only, replicated ledger; the integrity, privacy, and verifiability properties examined in this study derive from the cryptographic protocol layered on it, together with its implementation, access control, and operational procedures, not from the ledger alone. system would behave under conditions representative of Philippine elections. Existing systems have been evaluated in other contexts, at other scales, and with different cryptographic designs, and no single study has characterized the integrity, privacy, security, and performance of an integrated cryptographic voting framework on a permissioned blockchain at Philippine election scales. This study addresses that absence by designing, implementing, and evaluating such a framework, named BalotaChain and built on the Saksi cryptographic framework, in order to generate the baseline data needed to inform future discussion of cryptographic voting in the Philippines.

Statement of the Problem

Empirical evidence on how a cryptographic voting framework on a permissioned blockchain would perform in Philippine elections, with respect to integrity, privacy, security, and performance, remains limited and design-specific, and findings obtained under other designs may differ under the integrated design studied here. This gap constrains further research in this area and requires further investigation.

Specifically, this study seeks to answer the following questions:

1. To what extent does the proposed framework operate correctly when evaluated as an election system at multiple scales, as measured by six criteria: (a) the announced tally equals the ground-truth tally; (b) invalid ballots are rejected; (c) duplicate votes are rejected; (d) all accepted ballots are included in the tally; (e) the result is independently verifiable from the public election record; and (f) all cryptographic proofs verify?

2. What integrity, privacy, and security properties does the proposed system demonstrate under specified threat conditions?
3. What performance characteristics does the proposed system show across election sizes representing selected Philippine electoral scales, and how do these compare to existing cryptographic voting systems on the same blockchain platform?

Objectives

The general objective of this study is to generate empirical data on how a cryptographic voting framework on a permissioned blockchain performs in Philippine elections with respect to integrity, privacy, security, and performance, by designing, implementing, and evaluating BalotaChain on the Saksi Framework.

Specifically, this study aims to:

- Implement the Saksi cryptographic framework as an integrated election system, comprising the cryptographic core, the permissioned blockchain layer, the voter application, the trustee application, and the open-source verifier, and verify its correct end-to-end operation across multiple election scales using synthetic voter populations;
- Evaluate the integrity, privacy, and security properties of the system by implementing concrete attacks for each defined adversary class and demonstrating resistance to the defined attack scenarios under the stated threat model while preserving voter-ballot unlinkability and end-to-end verifiability; and
- Characterize the performance of the system by benchmarking it across election scales and ballot complexities on measures of latency, throughput, threshold-decryption time, and resource consumption, and comparing the results against an existing cryptographic voting system on the same blockchain platform.

Scope and Limitations

This study covers the design, implementation, and evaluation of the Saksi cryptographic framework and its instantiation as the BalotaChain voting system. The evaluation will be conducted with synthetic voter populations via simulation than a live election, and the study does not deploy the system in any real electoral exercise. The evaluation is organized along two axes: election scale, ranging from one thousand voters, the maximum clustered precinct size under COMELEC rules, to one million voters as the largest measured tier, culminating in two capstone tiers of 1,921,917 voters, the combined electorate of the three Zamboanga Peninsula provinces, and 3,524,078 voters, the full ZAMBASULTA electorate; the capstone attempts are a committed part of the evaluation, and each outcome, completion or a measured scaling limit, is reported as a primary finding of the study, with tiers anchored to actual Mindanao electoral units, and ballot complexity, distinguishing single-position ballots from multi-position ballots representative of Philippine elections.

The single-position configuration is used to directly compare results with an existing baseline system on the same blockchain platform, so that the performance comparison reflects differences in cryptographic design rather than differences in workload. The multi-position configuration, modeled on a representative three-position ballot, simulates the system under realistic Philippine ballot conditions. Because the data are synthetic and generated by the researchers, the study involves no human participants and collects no personal information.

The study is limited to producing baseline data on integrity, privacy, security, and performance, and does not constitute a deployment, a security certification, or a feasibility study of conducting a national election with the proposed system. The benchmarking is performed on the researchers' available hardware, and absolute performance figures are specific to that environment.

Significance of the Study

This study contributes the empirical baseline data that does not currently exist on the integrity, privacy, security, and performance of a cryptographic voting framework on a permissioned blockchain at scales representative of Philippine elections. By providing this baseline characterization data, the study replaces assumption with evidence as the basis for future research on cryptographic voting in the Philippine context.

For researchers, the study provides a reusable cryptographic framework that is designed to be independent of the election domain and potentially applicable to other fields requiring verifiable, privacy-preserving aggregation, an open-source verifier that can serve as a foundation for further work, and a benchmark against which future systems can be measured.

For election administrators and policymakers, it offers concrete evidence regarding the behavior and resource requirements of such a system.

For the broader public, the study advances the goal of elections whose correctness can be independently verified rather than merely trusted, which is of particular relevance to the ongoing pursuit of electoral integrity in the Philippines.

Definition of Terms

The following terms are defined as they are used in this study:

End-to-End Verifiability. A property of a voting system in which a voter can confirm that their vote was recorded as cast and counted as recorded, and any observer can confirm that the announced tally is correct, without revealing how any individual voted.

Permissioned Blockchain. A distributed ledger whose participating nodes are known and authorized, as opposed to a public blockchain open to

anonymous participants. In this study, Hyperledger Fabric serves as the permissioned blockchain platform.

Threshold ElGamal Encryption. A form of public-key encryption in which the capability to decrypt is divided among several parties, so that decryption requires the cooperation of a minimum number, or threshold, of them, and no single party can decrypt alone.

Distributed Key Generation. A cryptographic procedure by which multiple parties jointly produce a public key while each holds only a share of the corresponding private key, ensuring that the complete private key never exists in any single location.

Homomorphic Aggregation. A property of certain encryption schemes that allows encrypted ballots to be combined into a single encrypted tally without decrypting any individual ballot, so that only the final aggregate result is ever decrypted.

Zero-Knowledge Proof. A cryptographic method by which one party can prove that a statement is true without revealing any information beyond the truth of the statement itself, used in this study to prove that a ballot is well-formed and that decryption was performed honestly.

Blind-Signed Anonymous Credential. A credential that authenticates a voter as eligible while preventing any party, including the issuer, from linking the credential to the voter's submitted ballot, thereby preserving voter-ballot unlinkability.

Bulletin Board. The public, append-only record of an election, on which encrypted ballots and cryptographic proofs are posted so that the conduct and outcome of the election can be independently verified. In this study, the permissioned blockchain serves as the bulletin board.

Trustee. One of the parties who jointly hold the election key and who cooperate, in a threshold arrangement, to decrypt the final aggregate tally. No group of trustees smaller than the threshold can decrypt.

Synthetic Voter Population. A set of artificially generated voter records and ballots, produced by the researchers for the purpose of evaluating the system, with known ground-truth vote counts and containing no real personal information.

Saksi Framework. The reusable cryptographic framework developed in this study, which provides the underlying cryptographic primitives and is independent of any particular election application.

BalotaChain. The end-to-end verifiable voting application developed in this study as a demonstration of the Saksi Framework, comprising the voter application, trustee application, blockchain chaincode, and verifier.

Blockchain. A distributed, append-only ledger in which records are grouped into cryptographically linked blocks, making prior entries tamper-evident. In this study, the blockchain serves as the public bulletin board that records encrypted ballots and proofs.

Smart Contract (Chaincode). Program logic that executes on the blockchain to enforce rules automatically. In this study, the chaincode verifies ballots, performs aggregation, and coordinates decryption on Hyperledger Fabric, where such programs are termed chaincode.

Hyperledger Fabric. An open-source permissioned blockchain platform used in this study as the substrate for the bulletin board, chosen for its access control, throughput, and suitability for elections among known authorities.

Cryptographic Voting. An approach to elections in which integrity and privacy are guaranteed by mathematical and cryptographic mechanisms rather than by procedural trust in the administrators.

Ballot Secrecy. The property that the content of any individual voter's ballot cannot be read by any party at any point in the election.

Unlinkability. The property that no party can associate a particular voter with their particular submitted ballot, even given full access to the public record.

Tally. The aggregate count of votes per candidate or option that constitutes the election result. In this study, no individual ballots are being decrypted for tally, only the aggregated value.

Adversary Model. The defined set of assumptions about which parties may behave maliciously and what capabilities they hold to try and exploit vulnerabilities in the system, against which the system's security is evaluated.

Nullifier. A single-use value that marks a credential as having voted for a given position, to prevent double voting while preserving the voter's anonymity.

Verifier. Open-source software that allows any independent party to re-check the cryptographic proofs and confirm the correctness of the election without trusting the administrators.

Saksi Framework. The application-agnostic cryptographic library developed in this study, providing encryption, distributed key generation, proof generation and verification, credentials, nullifiers, partial decryption, and tally combination. The name is used only for the library, never for the election application.

BalotaChain. The election application built on the Saksi framework and evaluated in this study; the term refers to the complete voting system, including the voter application, trustee console, chaincode, and verifier.

Bulletin Board. The public, append-only record of the election. In this study the Hyperledger Fabric ledger serves as the bulletin board; the terms refer to the same artifact viewed as a cryptographic abstraction and as a platform component, respectively.

Trustee. One of the five independent parties holding a share of the election private key; any three may jointly decrypt the aggregate, and no smaller group can.

Election Authority. The administrative role that publishes the election manifest and operates registration; it holds no decryption capability.

Election System and Voting System. Used interchangeably for the complete deployed arrangement of software, protocol, and procedures; voting framework is not used except in reference to the Saksi framework as defined above.

CHAPTER II

REVIEW OF RELATED LITERATURE

2.1 Introduction

Trustworthy elections must satisfy two properties that are hard to reconcile. The first is integrity: every vote is counted exactly as cast. The second is privacy: no party can link a voter to the contents of their ballot. Paper ballots and early electronic voting machines achieve both through procedural safeguards and institutional trust, relying on observers, custody chains, and the assumed honesty of election officials. Over the past two decades, researchers have worked to replace that procedural trust with mathematical guarantees. This effort produced the field of cryptographic voting and, more recently, blockchain-based voting [1], [2].

This shift is most relevant where public trust in the electoral process is contested. The Philippines has run nationwide automated elections since 2010 using optical-scan technology. That experience shows large-scale electronic vote counting is practical, but it has also drawn repeated concerns about transparency and auditability [3]. The present study responds to those concerns. It asks whether a cryptographic voting framework on a permissioned blockchain can provide verifiable integrity and privacy guarantees suitable for the Philippine setting.

This chapter reviews the literature in five parts. The first covers the cryptographic primitives that underlie verifiable voting systems. The second

surveys the main end-to-end verifiable systems that combine those primitives into full protocols, and the third turns to the newer work that uses a blockchain as the bulletin board. The fourth examines the formal security models that define verifiability and coercion resistance. The fifth reviews real deployments, which show both what electronic voting can do in practice and where it falls short. The chapter closes by drawing together the limits of existing approaches and stating the gap this study addresses.

2.2 Cryptographic Foundations of Verifiable Voting

Verifiable voting systems are built from a small set of cryptographic primitives. Combined, these let votes be encrypted, counted, and checked for correctness without exposing any individual ballot. The first is the public-key cryptosystem of ElGamal [4], whose security depends on the hardness of the discrete-logarithm problem. ElGamal encryption is homomorphic: the product of two ciphertexts decrypts to the sum of their plaintexts. This is what lets encrypted ballots be combined into an encrypted tally without decrypting any single vote, which is why homomorphic voting schemes are built on it.

A second primitive addresses the danger of concentrating decryption power in a single authority. Pedersen [5] introduced a non-interactive and information-theoretically secure verifiable secret-sharing scheme, together with the commitment scheme that bears his name. This construction allows a secret, such as an election decryption key, to be distributed among several parties such that no individual holds the complete key, while still permitting verification that the shares are consistent. It forms the basis of distributed key generation, in which a group of trustees jointly produces an election key whose private component never exists in one place.

The remaining two primitives provide verifiability through zero-knowledge proofs, which allow a party to prove that a statement is true without revealing any information beyond the truth of the statement itself. Chaum and Pedersen [6]

introduced a proof of equality of discrete logarithms, which enables a trustee to prove that a ciphertext was decrypted correctly with respect to a committed key share, without disclosing the share. Cramer, Damgård, and Schoenmakers [7] generalized this idea into proofs of partial knowledge, commonly known as disjunctive proofs, which allow a prover to demonstrate that a ciphertext encrypts one of a set of valid values, such as a zero or a one, without revealing which. In a voting context, this is precisely the proof that a submitted ballot is well-formed and represents a valid selection. Together, these four primitives establish the pattern that recurs throughout the verifiable-voting literature: encrypt each vote, aggregate the ciphertexts homomorphically, prove well-formedness and correct decryption in zero knowledge, and decrypt only the final tally through a distributed set of trustees. Figure 2.1 illustrates how these primitives compose into the layers of a complete voting system.

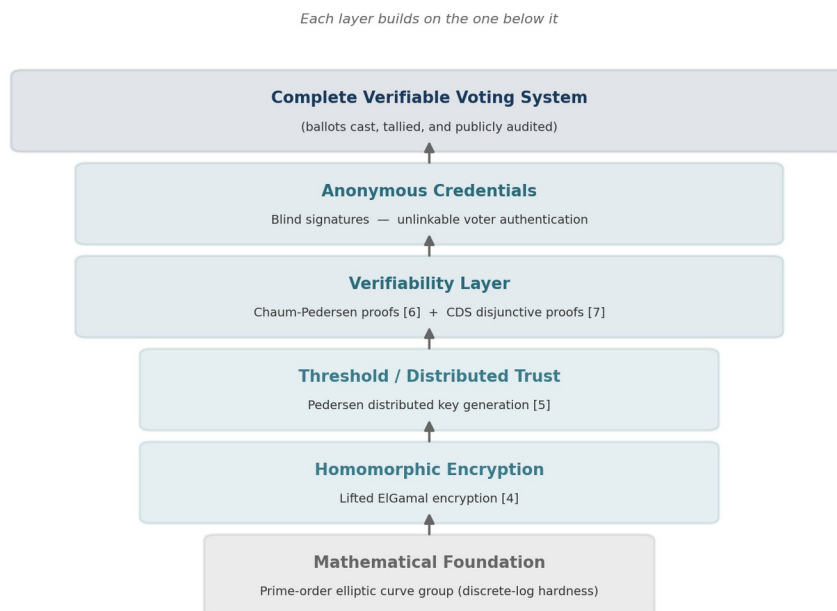


Figure 2.1. Layered composition of cryptographic primitives in a verifiable voting system.

2.3 End-to-End Verifiable Voting Systems

The composition of these primitives into complete protocols gave rise to end-to-end verifiable voting systems, in which voters can confirm that their vote was recorded as cast and counted as recorded, and any observer can confirm that the announced result is correct. Figure 2.2 situates the major systems and primitives reviewed in this chapter along a timeline, showing the progression from foundational primitives, through the first generation of verifiable systems, to the recent blockchain-based era.

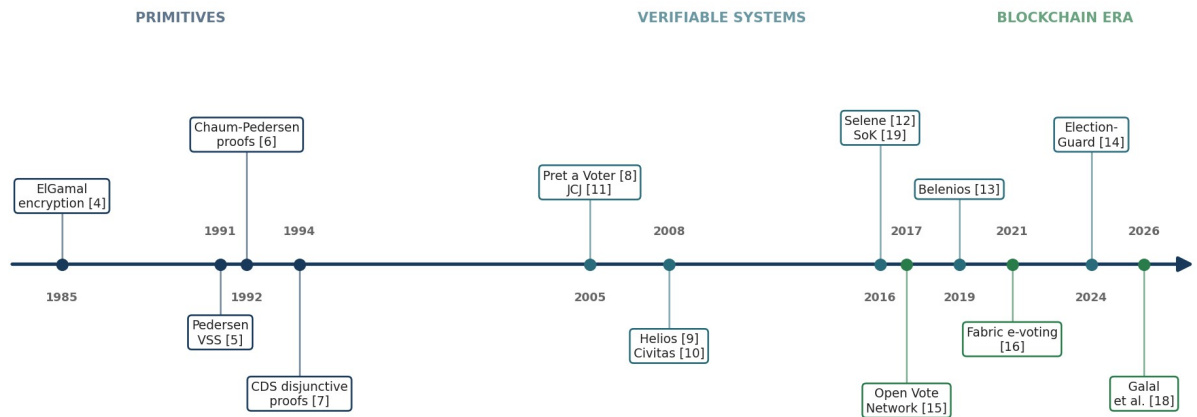


Figure 2.2. Timeline of key developments in cryptographic and blockchain-based voting (1985–2026).

An early and influential design in this space is *Prêt à Voter*, proposed by Chaum, Ryan, and Schneider [8], which applies end-to-end verifiability to paper ballots by giving voters an encrypted receipt that reveals nothing about their selection yet allows them to confirm their ballot's inclusion. The scheme demonstrated that verifiability need not depend on trusting the voting equipment, though it carries the logistical overhead of secure ballot printing.

The first widely used web-based system was Helios, introduced by Adida [9]. Helios combines homomorphic ElGamal encryption with zero-knowledge

proofs to offer open-audit voting over the internet, and it has run binding elections such as university and association governance. Its main limits come from a centralized design. One server runs the bulletin board, and one trustee holds the decryption key. As a result, Helios gives verifiability but not distributed trust, and it does not protect against voter coercion. Civitas, developed by Clarkson, Chong, and Myers [10], addressed coercion directly and was the first working coercion-resistant voting system. It builds on the formal coercion-resistance model of Juels, Catalano, and Jakobsson [11], and uses anonymous credentials that let a coerced voter deceive the coercer while still casting a valid vote. Civitas offers strong, well-proven security, but at a heavy computational cost that limits its scalability.

Later systems refined the trade-off between verifiability, usability, and coercion resistance. Selene, by Ryan, Ronne, and Iovino [12], added a tracker-number scheme that lets each voter find their own vote in plaintext on the bulletin board after the election. This gives an intuitive kind of verifiability, and its deniable tracker assignment blunts coercion. Belenios, by Cortier, Gaudry, and Glondou [13], extended the Helios line by authenticating voters with digital signatures, which gives eligibility verifiability and blocks ballot stuffing. Its main limit is that the link between a voter and their signed ballot stays visible on the bulletin board, and it depends on a trusted registrar. The broadest recent system is ElectionGuard, by Benaloh, Naehrig, Pereira, and Wallach [14]. It is a cryptographic toolkit that combines threshold ElGamal encryption, distributed key generation, and disjunctive proofs of ballot well-formedness, and it has run in public elections in several US jurisdictions. ElectionGuard leaves the choice of bulletin-board substrate to the deployer, and its documentation lists the practical independence of its trustees, called guardians, as an open problem.

2.4 Blockchain-Based Voting Systems

A parallel line of research uses blockchain as the bulletin board for voting, drawn by its decentralization, immutability, and public verifiability. An early and

often-cited example is the Open Vote Network of McCorry, Shahandashti, and Hao [15], which the authors describe as the first decentralized, self-tallying internet voting protocol with maximum voter privacy, built as a smart contract on Ethereum. It is an elegant design, but it targets boardroom-scale elections with few participants, not large public ones, and it carries the transaction costs of a public blockchain.

Research has since shifted to permissioned blockchains, which limit participation to known parties and so allow higher throughput and more governance control. Díaz-Santiso and Fraga-Lamas [16] built an electronic voting system on Hyperledger Fabric and measured its latency and transaction load with the Hyperledger Caliper benchmarking tool, reporting good performance. Pereira and colleagues [17] proposed a blockchain voting system focused on security and transparency, built on a custom chain. It shows the properties they wanted, but resting on a bespoke chain instead of a mature, audited platform leaves its production readiness in doubt. The closest recent work to this study is by Galal, El Reheem, and Guirguis [18], who built a voting system on Hyperledger Fabric using AES-256 for ballot content and RSA-2048 for key protection, then stress-tested it with synthetic populations of up to fifty thousand voters. Their results show that Hyperledger Fabric can handle elections at this scale. But their design puts all decryption authority in a single election commission that holds the RSA private key, a centralization their own analysis flags as a serious risk. Surveys of this fast-growing field, including Vladucu and colleagues [1] and the systematic review by Jafar and colleagues [2], map the design space and keep pointing to the same two open problems: scalability, and squaring transparency with privacy.

2.5 Security Models and Verifiability Definitions

Underlying the systems described above is a body of theoretical work that defines precisely what their security claims mean. The notion of coercion resistance was formalized by Juels, Catalano, and Jakobsson [11], who modeled

an adversary capable of coercing voters or buying votes and defined the conditions under which a voting system can resist such an adversary; their anonymous-credential construction has influenced nearly every subsequent coercion-resistant design. The various and sometimes inconsistent definitions of verifiability that appeared across the literature were later unified by Cortier, Galindo, Küsters, Müller, and Truderung [19], whose systematization of knowledge distinguishes individual verifiability, universal verifiability, and eligibility verifiability within a single formal framework. These theoretical contributions provide the vocabulary necessary to state and evaluate the security guarantees of any voting system rigorously, rather than relying on informal assurances.

2.6 Electronic Voting in Practice

Beyond the lab, a few jurisdictions have run electronic voting at national scale, which gives real evidence of both what works and what does not. Estonia is the best-known case, with binding nationwide internet voting since 2005. The long-term study by Ehin, Solvak, Willemson, and Vinkel [20], covering eleven elections from 2005 to 2019, tracks how internet voting grew steadily until it made up nearly half of all votes cast. That record shows sustained large-scale electronic voting is achievable when the infrastructure and public trust are there. For the Philippines, Reyes [3] analyzes the 2010 automated presidential election and the move to optical-scan counting, and finds that automation changed how the process ran but left open questions about the transparency and auditability of the systems behind it. Taken together, these cases show that electronic voting works in practice, but that the reliance on proprietary vendor systems and the lack of end-to-end cryptographic verifiability are still serious concerns.

[21] Commission on Elections, “Consolidated 2025 Number of Registered Voters,” Records and Statistics Division, Republic of the Philippines, 2025. [Online]. Available: <https://comelec.gov.ph/?r=2025NLE/Statistics>

- [22] Commission on Elections, Resolution No. 10965, "Guidelines for the Clustering of Established Precincts in Connection with the May 12, 2025 National and Local Elections," Republic of the Philippines, 2024.
- [23] Hyperledger Performance and Scale Working Group, "Hyperledger Blockchain Performance Metrics," White Paper v1.01, The Linux Foundation, 2018.
- [24] Hyperledger Foundation, "Benchmarking Hyperledger Fabric 2.5 Performance," LF Decentralized Trust, Feb. 2023. [Online]. Available: <https://www.lfdecentralizedtrust.org/blog/2023/02/16/benchmarking-hyperledger-fabric-2-5-performance>
- [25] U.S. Election Assistance Commission, "Voluntary Voting System Guidelines VVSG 2.0," Feb. 2021.
- [26] Y.-N. Jung, S.-B. Cho, S.-H. Yun, and H.-J. Seo, "Anonymous E-Voting on Hyperledger Fabric," Cryptology ePrint Archive, Paper 2026/1176, 2026. [Online]. Available: <https://eprint.iacr.org/2026/1176>
- [27] A. Pfitzmann and M. Hansen, "A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management," v0.34, Aug. 2010. [Online]. Available: https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf
- [28] R. L. Rivest, "On the notion of 'software independence' in voting systems," Philosophical Transactions of the Royal Society A, vol. 366, no. 1881, pp. 3759–3767, 2008, doi: 10.1098/rsta.2008.0149.
- [29] D. Shanks, "Class number, a theory of factorization, and genera," in Proceedings of Symposia in Pure Mathematics, vol. 20, American Mathematical Society, 1971, pp. 415–440.
- [30] International Foundation for Electoral Systems, "Briefing Paper: Republic Act 9369 and the Automated Election System," IFES Philippines, 2010. [Online].
- [31] International IDEA, "Legal Challenges to Election Technology: The Case of the Philippines," International Institute for Democracy and Electoral Assistance, 2020. [Online].

2.7 Synthesis and Research Gap

Looked at side by side, these approaches share one trait: each solves only part of the problem. The cryptographic primitives in Section 2.2 give all the pieces needed for verifiable, private voting, but on their own they are not full systems. The end-to-end verifiable systems in Section 2.3 each add something useful and each leave a clear gap. Helios [9] centralizes both the bulletin board and the decryption authority. Belenios [13] exposes the link between a voter and their ballot on the public bulletin board. ElectionGuard [14] has the most complete cryptographic stack, but it leaves the choice of bulletin-board substrate to the deployer and leaves trustee independence unsolved in practice. The blockchain-based systems of Section 2.4 address the bulletin-board concern through decentralization, but the Open Vote Network [15] operates only at boardroom scale, the proposal of Pereira and colleagues [17] remains a conceptual design analysis without a measured implementation, and the work of Galal and colleagues [18], while demonstrating large-scale feasibility on a mature permissioned platform, concentrates decryption authority in a single party and employs an encryption scheme that cannot support homomorphic aggregation or threshold decryption. The formal frameworks of Section 2.5 complete the picture from the opposite direction: they supply the definitions of verifiability and privacy used throughout this study, but they build and measure no systems. And the deployment studies of Section 2.6, from the Philippine automation experience of 2010 [3] to the eleven Estonian internet elections [20], establish that electronic voting can be operated and adopted at national scale, while leaving the cryptographic verifiability of the deployed systems outside their scope. Practice has thus run ahead of verifiable cryptography, and verifiable cryptography has run ahead of deployment-scale evidence.

The precise gap that emerges is the absence of an integrated framework that combines threshold ElGamal encryption, distributed key generation across mutually independent trustees, zero-knowledge proofs of both ballot well-

formedness and correct decryption, blind-signed anonymous credentials for unlinkable voter authentication, and homomorphic aggregation, all deployed on a production-grade permissioned blockchain, and whose integrity, privacy, security, and performance properties have been characterized empirically at the scales representative of Philippine institutional and government elections, up to the full ZAMBASULTA electorate of 3,524,078 registered voters. No existing study provides this characterization. This gap matters because, in its absence, institutions considering the adoption of cryptographic voting must rely on assumptions rather than evidence, and researchers lack a baseline against which to measure further progress. The present study addresses this gap by designing, implementing, and evaluating such a framework, thereby providing the empirical data that does not currently exist.

Table 2.1 compares the most representative systems across the architectural dimensions that define the gap, and Figure 2.3 presents the same comparison as a feature-coverage matrix. Both make clear that while each existing system provides a subset of the required properties, none provides all of them simultaneously, and none has been evaluated at Philippine election scales.

Table 2.1. Comparison of representative voting systems across key architectural dimensions.

System	Substrate	Encryption	Decryption authority	Verifiability	Unlinkability
Helios [9]	Centralized server	Homomorphic ElGamal	Single trustee	End-to-end	Procedural
Belenios [13]	Centralized server	ElGamal + signatures	Trustee set	End-to-end + eligibility	Linkage on board
ElectionGuard [14]	Deferred to deployer	Threshold ElGamal	Guardian quorum	End-to-end	Not specified
Open Vote Net. [15]	Ethereum (public)	Self-tallying (DRE)	Self-tallying	End-to-end	Cryptographic
Galal et al. [18]	Hyperledger Fabric	AES-256 + RSA-2048	Single commission	Transparency	Procedural
Proposed (this study)	Hyperledger Fabric	Threshold ElGamal	3-of-5 trustees	End-to-end	Blind-signed credentials



Figure 2.3. Feature-coverage matrix of representative systems against the requirements identified in this review. Among the systems reviewed, only the proposed framework satisfies all seven properties.

2.8 Review Matrix

Table 2.2 summarizes the twenty studies reviewed in this chapter. Each technical claim recorded in the matrix has been checked against the primary source cited in its row, so that the basis of every reported limitation can be produced on request. The matrix compares the problem each addresses, the algorithm or method employed, the datasets used, the form of evaluation, and the principal strengths and limitations of each.

Table 2.2. Review matrix of related studies.

Ref.	Problem addressed	Algorithm / Method	Datasets	Evaluation	Strengths	Limitations
[1]	Map blockchain e-voting landscape	Survey (multiple systems)	N/A (review)	Qualitative comparison	Broad coverage; security taxonomy	No implementation; survey only
[2]	Scalability of blockchain voting	Systematic review + meta-analysis	76 articles, 2017-2022	Systematic review	Rigorous method; scalability focus	Secondary data; no new system
[3]	Automation impact on PH	Case study (qualitative)	2010 PH presidential	Qualitative case study	Philippine context; real deployment	Political not cryptographic; dated

	elections		election			
[4]	Public-key encryption from discrete log	ElGamal encryption / signature	N/A (theoretical)	Security proof	Homomorphic; foundational	Not a voting system itself
[5]	Verifiable secret sharing without trusting the dealer	Pedersen VSS / commitments	N/A (theoretical)	Information-theoretic proof	No single key holder; hiding	Primitive only
[6]	Prove correct decryption privately	Chaum-Pedersen ZK proof (DLEQ)	N/A (theoretical)	ZK proof of equality of discrete logs	Verifiable decryption	Single-statement primitive
[7]	Prove partial knowledge (OR proofs)	CDS disjunctive proofs	N/A (theoretical)	Witness-hiding proof	Ballot well-formedness proofs	Primitive only
[8]	Voter-verifiable paper ballots	Encrypted receipts + mix-net	N/A (scheme)	Scheme analysis	End-to-end paper voting; receipts	Paper logistics; printing trust
[9]	Web-based open-audit voting	Homomorphic ElGamal + ZK proofs	Prototype web deployment	Deployment + open audit	First practical web E2E system	Single trustee; centralized server
[10]	Coercion-resistant remote voting	JCJ credentials + mix-net	Performance simulation	Implementation + scalability test	First coercion-resistant impl.	Complex; high performance cost
[11]	Define and achieve coercion resistance	Anonymous credentials	N/A (theoretical)	Formal model + scheme	Formal coercion model	Quadratic tally cost
[12]	Transparent verifiability + coercion mitigation	Tracker numbers + ElGamal	N/A (scheme)	Scheme analysis	Voter sees plaintext vote	Tracker management; mitigation not full resistance
[13]	Verifiable voting + authentication	ElGamal + digital signatures	Real elections (Belenios)	Deployment	Eligibility verifiability; anti-stuffing	Voter-ballot linkage on board; trusted registrar
[14]	Reusable verifiable election toolkit	Threshold ElGamal + DKG + disjunctive proofs	US public elections	Real deployments	Threshold; toolkit; deployed	Bulletin board deferred; guardian independence open
[15]	Decentralized self-tallying voting	Smart contract + Open Vote Network	Ethereum testnet (40 voters)	Gas cost / feasibility	Decentralized; maximum privacy	Boardroom scale only; Ethereum cost
[16]	E-voting on permissioned blockchain	Hyperledger Fabric + smart contracts	Caliper benchmark	Latency / throughput benchmark	Fabric platform; benchmarked	Limited cryptography; prototype
[17]	Secure transparent blockchain voting	Digital signatures + homomorphic encryption + ZKPs on BFT consensus (design)	N/A (conceptual design)	Design analysis	Transparency; decentralization	Conceptual; no measured implementation
[18]	Privacy +	AES-256 + RSA-	Synthetic, up	Performance	Fabric; large-scale	Centralized RSA

	transparency at scale	2048 on Hyperledger Fabric	to 50,000 voters	stress test	benchmark	decryption; no threshold or homomorphic tally
[19]	Unify verifiability definitions	KTV framework (theoretical)	N/A (theoretical)	Systematization of knowledge	Formal verifiability taxonomy	Theoretical; no system
[20]	National internet voting adoption	Deployment study	11 Estonian elections, 2005-2019	Longitudinal empirical study	Real nationwide data; longitudinal	Context-specific; not cryptographic analysis

2.9 Conceptual Framework

The conceptual framework of this study, shown below, instantiates the input-process-output logic with the specific content of this research: the Philippine electoral environment supplies relevance, the cryptographic knowledge base supplies rigor, the protocol chain from registration to independent verification constitutes the process, and a design and evaluation feedback loop closes the cycle. It follows directly from the synthesis and research gap established in the preceding sections. The framework positions the proposed system as the process that transforms defined inputs into the measurable outputs that constitute the contribution of the study. The detailed design of the proposed framework itself is presented in Chapter III.

The input consists of validated synthetic voter populations, each carrying known per-position ground-truth vote counts, generated across two axes: election scale, defined as the number of voters, and ballot complexity, defined as the number of positions on the ballot. The input also includes the threat and adversary conditions under which the system is evaluated. The process is the operation of the cryptographic framework as an end-to-end verifiable election, encompassing encryption, zero-knowledge proof generation, homomorphic aggregation, threshold decryption, and verification on a permissioned blockchain. The output is an empirical characterization of the system across four properties. Accuracy shows integrity and is measured against known ground truth. Privacy is tested structurally, through adversarial unlinkability testing. Security is judged

against defined adversary classes. Performance is measured quantitatively. This gives baseline data on accuracy, privacy, security, and performance that does not yet exist for cryptographic voting in the Philippines, which is exactly what the research gap in this chapter calls for.

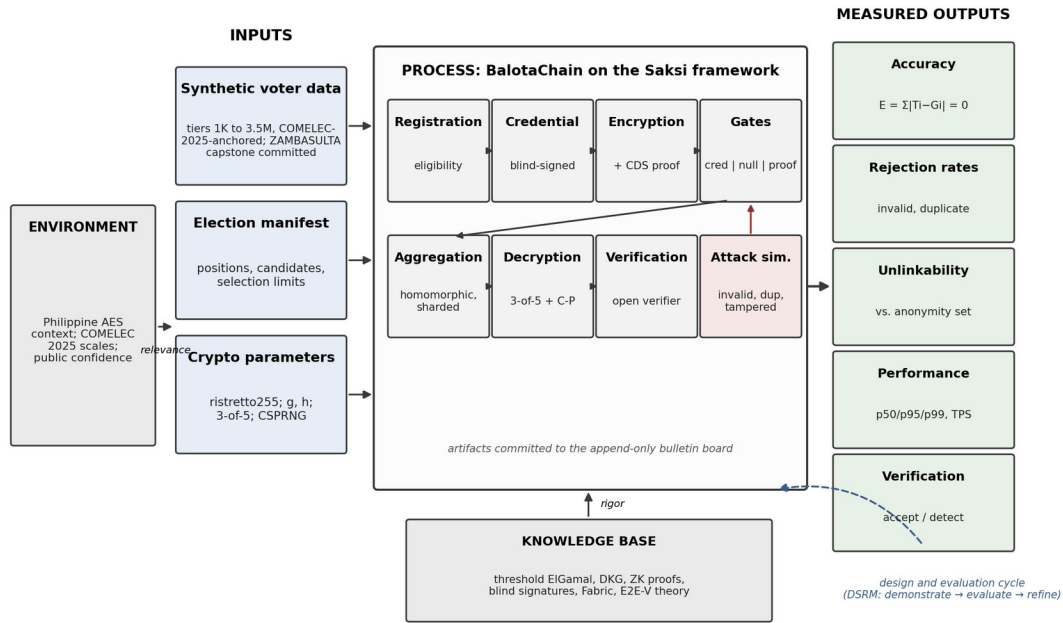


Figure 2.4. Conceptual framework of the study: inputs, the Saksi and BalotaChain process chain, measured outputs, and the design and evaluation cycle.

CHAPTER III

METHODOLOGY

Research Design

This study uses Design Science Research (DSR) as its methodology, following the DSRM process model of Peffers and colleagues. DSR fits because the study's main output is an artifact: the Saksi cryptographic framework and its implementation as the BalotaChain voting system. The knowledge contribution is the empirical characterization of that artifact. Among the usual research designs in computer science, this work is applied and simulation-based. It applies established cryptographic primitives to build a working system, and it evaluates that system by simulation with synthetic voter populations instead of a live election. The DSRM framing puts these activities in a clear sequence: problem identification, objective definition, design and development, demonstration, evaluation, and communication. Iteration is allowed when evaluation shows the artifact needs more work.

Figure 3.1 presents an overview of the research methodology from problem identification through evaluation. The process begins by identifying the absence of empirical data on cryptographic voting on permissioned blockchain in the Philippine context, and defines the objective of producing a framework that provides verifiable accuracy, privacy, and security together with the data that characterizes these properties. The artifact is then designed and developed across its components: the Saksi cryptographic core in Rust, the Hyperledger Fabric chaincode, the Flutter voter application, the Tauri trustee application, and the open-source verifier. Before any demonstration occurs, synthetic voter populations and their known vote-to-candidate ground-truth counts are generated and passed through a data validation gate; only data confirmed to be complete, well-formed, and internally consistent proceeds, while invalid data is regenerated or corrected. The validated data then drives a complete end-to-end election in

the demonstration stage, after which the system is evaluated for accuracy, privacy, security, and performance across two experimental axes: election scale, defined as the number of voters, and ballot complexity, defined as the number of positions contested on a single ballot. Findings are documented and communicated as the baseline data contribution. The figure also shows the Design Science iteration loop, in which shortcomings identified during evaluation feed back into design and development.

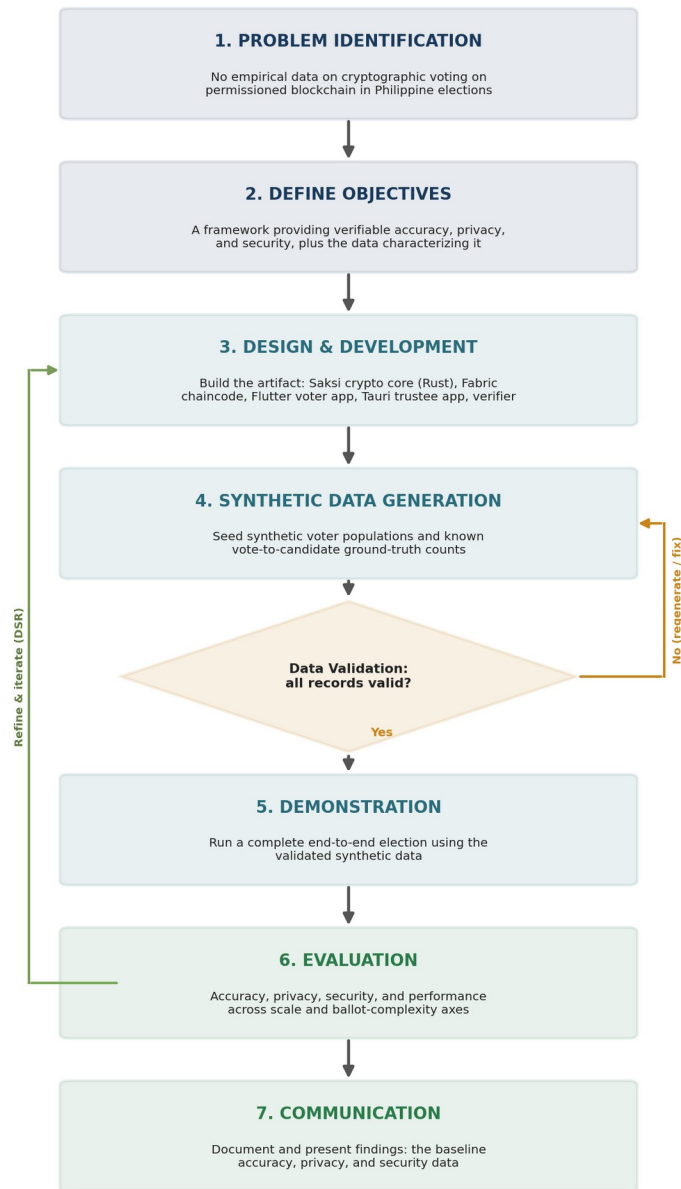


Figure 3.1. Overview of the research methodology following the DSRM process model.

Table 3.1 maps each phase of the DSRM process model to its concrete application in this study, so that the methodology is not merely cited but shown in the specific form it takes here. The phases are iterative rather than strictly

sequential: demonstration and evaluation feed back into design and development, which is the design and evaluation cycle shown in Figure 2.4.

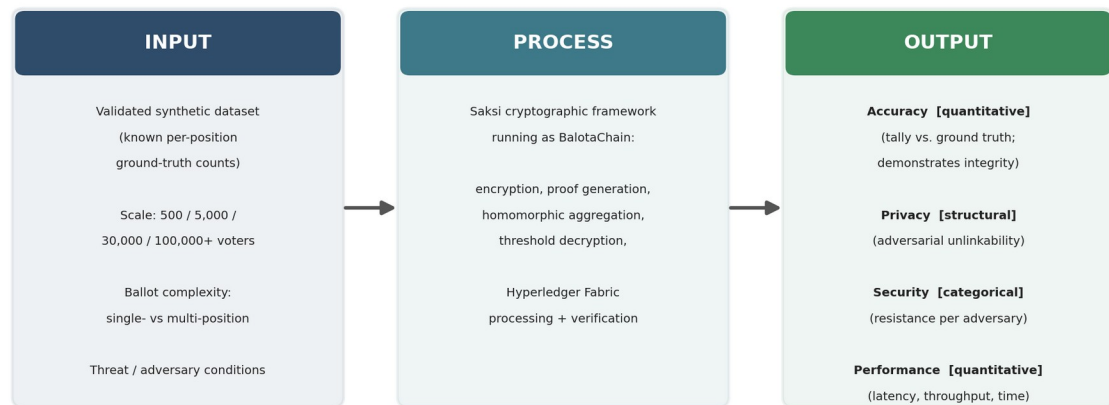
Table 3.1. Application of the DSRM process model to this study.

DSRM phase	Application in this study	Location in the manuscript
1. Problem identification and motivation	Empirical evidence on end-to-end verifiable, privacy-preserving elections on a permissioned blockchain at Philippine electoral scale remains limited and design-specific; the automated election system context and the public-confidence problem motivate the study.	Chapter I, Background of the Study and Statement of the Problem
2. Objectives of a solution	An application-agnostic cryptographic framework (Saksi) satisfying the seven properties identified in the review, and a first instantiation (BalotaChain) that demonstrates correctness, verifiability, unlinkability, and measured performance at defined scales.	Chapter I, Objectives of the Study; Chapter II, Figure 2.3
3. Design and development	Saksi provides threshold ElGamal encryption on ristretto255, Pedersen distributed key generation at a three-of-five threshold, CDS disjunctive proofs of ballot validity, Chaum-Pedersen proofs of honest partial decryption, blind-signed anonymous credentials, and deterministic nullifiers. BalotaChain instantiates these on Hyperledger Fabric with Raft ordering, Go chaincode enforcing the credential, nullifier, and proof gates, homomorphic aggregation, a Flutter voter application, a Tauri trustee console, and an open verifier.	Chapter III, The Saksi Framework and Software Development
4. Demonstration	The complete election lifecycle is executed on synthetic election workloads representing selected Philippine electoral scales, from one thousand voters to full ZAMBASULTA scale (3,524,078 voters), culminating in the committed capstone attempt at the full ZAMBASULTA electorate, including a worked multi-position election.	Chapter III, Experimental Design (Tables 3.7 and 3.8)
5. Evaluation	Accuracy is evaluated against ground truth; rejection behaviour, unlinkability, and resistance to the defined attack scenarios are measured; latency, throughput, and resource utilisation are reported with percentiles; the independent verifier reproduces the tally from the public record and a tampering trial tests detection; results are compared against the published baseline as a cross-study comparison.	Chapter III, Testing and Evaluation; Chapter IV
6. Communication	The findings are communicated through this manuscript and through the separately versioned open-source Saksi and BalotaChain repositories with build, test, and deployment documentation.	Whole manuscript; Chapter III, Software Development

Throughout this manuscript, statements of capability carry an explicit epistemic status: design intent for properties the architecture is intended to provide, implemented for mechanisms realized in code, and demonstrated for

results established by the experiments of this chapter. No property is claimed beyond the strongest status it has attained under the stated experimental conditions and threat model.

Figure 3.2 expresses the same study in input-process-output terms, clarifying what enters the system, what the system does, and what is measured. The inputs are the validated synthetic dataset, the voter populations across the two experimental axes of scale and ballot complexity, and the adversary conditions used in security testing. The process is the Saksi framework operating as BalotaChain, performing encryption, proof generation, homomorphic aggregation, threshold decryption, and verification on Hyperledger Fabric. The outputs are the measured properties: accuracy and performance as quantitative measures, and privacy and security as structural and categorical results.



Output is the empirical baseline data on accuracy, privacy, and security that does not currently exist for cryptographic voting in the Philippine context.

Figure 3.2. Input-Process-Output framework of the study.

The Saksi Framework

Saksi, the Filipino word for witness, is designed as an application-agnostic cryptographic framework for verifiable, privacy-preserving aggregation. BalotaChain is its first instantiation: the election application built on top of it, and

the vehicle through which the framework is evaluated in this study. The separation is deliberate. Saksi is a reusable, independently versioned library, written in Rust, that provides the cryptographic operations and makes no assumptions about any particular election, or about elections at all; BalotaChain supplies the election-specific workflow, the user-facing applications, and the deployment on the blockchain substrate. This division allows the cryptographic core to be tested and audited in isolation against published test vectors, and allows the same framework to be adopted by other applications that require the same guarantees.

The framework comprises five components, detailed in the Software Development section and Table 3.3: the Saksi cryptographic core, which implements threshold ElGamal encryption on ristretto255, Pedersen distributed key generation, CDS disjunctive and Chaum-Pedersen zero-knowledge proofs, and blind-signature credential issuance; the Hyperledger Fabric chaincode, written in Go, which operates the bulletin board and enforces the submission gates; the Flutter voter application; the Tauri trustee console; and the open-source verifier that allows any independent party to re-check the election from the public record alone.

Figure 3.3 summarizes the workflow as instantiated in BalotaChain. At setup, five trustees jointly generate the election key through Pedersen distributed key generation, so that no single party ever holds the whole private key and any three of the five are required to decrypt. At registration, eligible voters obtain blind-signed anonymous credentials that prove eligibility without linking a voter to a ballot. At voting, the voter application encrypts the selection under the election key and attaches a CDS disjunctive proof of ballot well-formedness together with a single-use nullifier. At submission, the chaincode independently verifies the credential, the nullifier, and the proof; only ballots that pass all three gates are committed to the bulletin board. Accepted ballots are homomorphically aggregated into an encrypted total without decrypting any individual ballot. At the tally ceremony, at least three trustees produce partial decryptions of the

aggregate, each accompanied by a Chaum-Pedersen proof of honest decryption, and Lagrange interpolation recombines the partial decryptions into the final tally. Finally, the verifier recomputes the aggregate and re-checks every proof from the bulletin board, so that the correctness of the outcome does not rest on trust in any operator.

Saksi cryptographic operations as instantiated in the BalotaChain election application

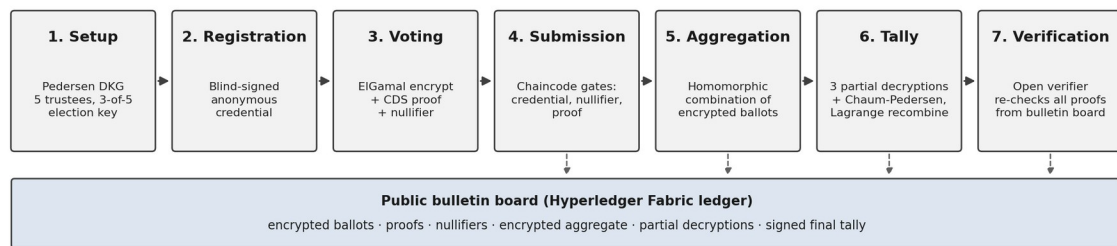


Figure 3.3. Workflow of the Saksi framework as instantiated in the BalotaChain election application, from key generation to independent verification.

Although this study exercises Saksi solely under election workloads, none of its primitives is specific to voting. The same combination of threshold encryption, distributed key generation, zero-knowledge validity proofs, blind-signed credentials, and homomorphic aggregation supports any setting in which individually sensitive inputs must be aggregated verifiably without being revealed, and in which no single party may hold the decryption capability. Two examples illustrate the design intent. In healthcare, encrypted patient responses or measurements could be aggregated into population statistics with a proof that each input is well-formed, without any individual record ever being decrypted. In sealed-bid auctions and public procurement, bids could remain encrypted until a threshold of independent parties jointly opens only the outcome, with each bid proven valid at submission. These applications are identified as design goals and directions for future work; evaluating Saksi outside the election domain is beyond the scope of this study, and no claim is made about its performance or suitability in those settings.

Data Collection

The data required by this study are not collected from human voters. Because the study characterizes the cryptographic and performance properties of a system, the data take the form of synthetic voter populations that serve as the test input to the framework, together with the measurements produced when the system processes that input. This approach follows common practice in cryptographic voting research, where synthetic populations are used to test system properties. Galal, El Reheem, and Guirguis, for example, benchmarked a Hyperledger Fabric voting system with synthetic populations of up to fifty thousand voters [18]. The synthetic data here are generated by code, not collected from any external source. That removes the need to recruit human subjects and the privacy exposure that comes with it, and it still exercises every cryptographic and blockchain operation in the system. Each synthetic record represents one vote cast by one voter for one position, and carries the attributes needed to drive the protocol: a unique voter identifier, the position being voted on, an anonymous credential issued during a simulated registration, a single-use nullifier, and a candidate selection drawn from a defined distribution. A voter participating in a multi-position ballot therefore produces one record per position, all sharing the same voter identifier but each with its own position, nullifier, and selection; double voting is prevented per voter per position rather than per voter overall. Because the selections are generated by the researchers, the true vote-to-candidate counts are known in advance for each position and constitute the ground truth against which the system's decrypted per-position tallies are later compared. The data are generated along two axes: election scale, the number of voters, at five tiers anchored to actual Mindanao electoral units drawn from official 2025 registration statistics [21]: one thousand, the maximum size of a clustered precinct under COMELEC Resolution No. 10965 [22]; ten thousand, a barangay-cluster scale; fifty thousand, a municipal scale that also matches the fifty-thousand-voter population measured by the baseline study [18]; four hundred eighty-three thousand, corresponding to Zamboanga City with 483,058

registered voters in the 2025 national and local elections; and one million, a provincial-scale stress test approaching Zamboanga del Sur, the largest Mindanao province at 1,196,013 registered voters. The ladder culminates in two capstone tiers: 1,921,917, the combined 2025 provincial electorate of Zamboanga del Norte (780,540), Zamboanga del Sur excluding Zamboanga City (712,955), and Zamboanga Sibugay (428,422); and 3,524,078, the full ZAMBASULTA electorate, adding Zamboanga City (483,058), Isabela City (83,477), Basilan (315,601), Sulu (464,393), and Tawi-Tawi (255,632), all figures from the 2025 COMELEC registration statistics. The capstone attempts are committed; the staged ladder below them is the engineering discipline that makes them reachable, and each capstone outcome, completion or a measured scaling limit, is reported as a primary finding of the study. The second axis is ballot complexity, the number of positions per ballot, ranging from a single position to a full multi-position ballot representative of Philippine elections. Table 3.2 summarizes the structure of a synthetic voter record; a sample of generated records and the generation parameters are provided in Appendix A.

Table 3.2. Structure of a synthetic voter record (data collection matrix).

Field	Description	Example / Source
voter_id	Synthetic identifier (repeats across positions)	V-000001 (generated)
position	Position being voted on	President / Senator / etc.
credential	Blind-signed anonymous credential	Issued in simulated registration
nullifier	Single-use token, one per voter per position	Derived from credential + position
selection	Chosen candidate (from defined distribution)	Candidate A / B / C / D
scale_group	Election size (number of voters)	1,000 / 10,000 / 50,000 / 483,000 / 1,000,000 / 1,921,917 / 3,524,078 (capstone)
ballot_complexity	Number of positions on the ballot	1 (baseline) ... full ballot
ground_truth_tally	Known per-position aggregate counts	Recorded at generation (for accuracy)

Before the data enter demonstration or evaluation, they pass through the data validation stage shown in Figure 3.1. A validation script checks that every record is complete and well-formed, that identifiers and nullifiers are unique, that selections fall within the valid candidate set, and that the aggregate ground-truth counts are internally consistent. Only data that pass these checks proceed; any record that fails is regenerated or corrected before the dataset is used. This stage makes sure the evaluation results reflect how the system behaves, not flaws in malformed input. It speaks directly to the feasibility and validity of the data the study depends on.

Algorithms

The Saksi framework combines several established cryptographic algorithms into one voting protocol. This section describes each algorithm and what it does. How they fit together into the system's layered architecture is shown in the figures of Chapter II and built out in the implementation under Software Development.

All cryptographic primitives operate over ristretto255, a prime-order group abstraction over Curve25519 with an approximately 252-bit scalar field; scalars and group elements are 32-byte encodings. Private keys and nonces are uniformly random scalars drawn from the operating system's cryptographically secure random number generator through the Rust `getrandom` interface. All zero-knowledge proofs are rendered non-interactive with the Fiat-Shamir transform over a domain-separated transcript hash, and a ciphertext is the pair of group elements $(c1, c2) = (g^r, g^m \cdot h^r)$, where g is the fixed group generator, h the election public key, m the encoded message, and r the encryption nonce. Pseudocode for each implemented primitive accompanies the released Saksi repository, with steps numbered exactly as implemented.

Algorithm 1: Threshold ElGamal Encryption on ristretto255

Ballots are encrypted with the ElGamal public-key cryptosystem [2], over the ristretto255 prime-order elliptic curve group. We chose ElGamal for its additive homomorphic property under a lifted encoding, which lets individual encrypted ballots combine into one encrypted tally without decrypting any single ballot. The encryption runs in threshold mode, so the decryption capability is split across several trustees instead of held by one party. Encoding and tally recovery proceed as follows: a selection is the bit b , encoded by lifting into the group as g^b before encryption; the component-wise product of all accepted ciphertexts for a contest yields $(g^{\sum r}, g^T \cdot h^{\sum r})$, where T is the number of votes for that selection; the trustees jointly remove the mask $h^{\sum r}$ through threshold partial decryption, leaving the group element g^T ; and the integer T is then recovered by a bounded discrete-logarithm search, a linear scan at the smaller tiers and Shanks's baby-step giant-step method [29] at fifty thousand voters and above, with the search bound set to the number of ballots accepted at that tier so that recovery is exact and always terminates.

Algorithm 2: Pedersen Distributed Key Generation

The election key pair comes from a distributed key generation ceremony based on Pedersen's verifiable secret-sharing scheme [3]. In this ceremony, five trustees jointly generate the election public key, and each one holds only a share of the matching private key. Decryption needs three of the five shares, so no single trustee, and no group smaller than three, can decrypt the tally. Trust is spread across independent parties, which removes the single point of compromise that single-authority designs have.

Algorithm 3: Zero-Knowledge Proofs

The system uses two families of zero-knowledge proofs. At ballot submission, a disjunctive proof of partial knowledge, based on the construction of Cramer, Damgard, and Schoenmakers [4], shows that an encrypted ballot holds

exactly one valid candidate without revealing which one. At tally time, a Chaum-Pedersen proof of equality of discrete logarithms [5] lets each trustee prove that its partial decryption was computed honestly from its committed key share, again without revealing the share. Together, the two proofs make the correctness of both ballot formation and decryption publicly verifiable.

Algorithm 4: Blind-Signed Anonymous Credentials

Voter authentication uses blind-signed anonymous credentials. At registration, the registrar signs a blinded value the voter supplies, so the voter gets a valid credential and the registrar never learns the value that will later identify the ballot. This verifies eligibility while keeping a voter and their submitted ballot unlinkable. Systems that record voter-ballot associations on the public bulletin board lack that unlinkability, which prior work has flagged as a limitation [6]. Ballots are multi-position: the election manifest defines one contest per position with its candidate list and selection limit, each contest carries its own encrypted accumulator on the ledger, and the nullifier is derived per credential and position through a domain-separated hash, so a voter may cast at most one accepted record per position while contests remain cryptographically independent. As a worked example, consider a manifest with three positions and five voters: each voter submits three encrypted records, one per contest, each with its own validity proof and nullifier; the chaincode gates admit fifteen records; aggregation runs per contest over its five ciphertexts; and threshold decryption of the three accumulators yields three independent tallies whose sums equal five, matching the seeded ground truth. At multi-million scale the per-contest accumulator becomes the design's hot key: every accepted ballot read-modify-writes the same ledger entry, so concurrent endorsement collapses into serialized commits. The chaincode therefore maintains per-precinct sub-accumulators, keyed by contest and precinct group, which are folded into the contest total in a single closing transaction after voting ends; correctness is unchanged because ciphertext addition is associative and commutative, and the fold is itself recorded on the bulletin board and re-checked by the verifier. Sub-

accumulator sharding is required for the capstone tiers and reduces write contention at every measured tier as well.

Software Development

The research output is a software system, and its development keeps the cryptographic framework clearly separate from the application built on it. Saksi is an independent, open-source Rust library with a public interface for encryption, distributed key generation, proof generation and verification, partial decryption, and tally combination. BalotaChain is a separate application that uses Saksi as an external dependency, which shows the framework is reusable rather than tied to one application. The split holds at the repository level: Saksi is versioned and tested on its own against published correctness vectors. Each repository documents its version tags, dependency manifests, build instructions, test commands, and deployment steps, so that any result reported in this study can be traced to the exact commit that produced it. Development proceeds in six increments: the cryptographic core, the election protocol, the blockchain integration, the client applications, testing, and evaluation. The complete protocol is validated end to end at one, ten, one hundred, and one thousand voters before any larger tier is attempted, and a failed stress tier is reported as a scaling-limit finding rather than treated as a failure of the study.

Figure 3.4 shows the component architecture and its trust boundaries: what runs on the client side, what the Saksi library provides, what executes inside the Fabric network, which data are public, and how the verifier operates entirely outside every trust boundary using the public record alone.

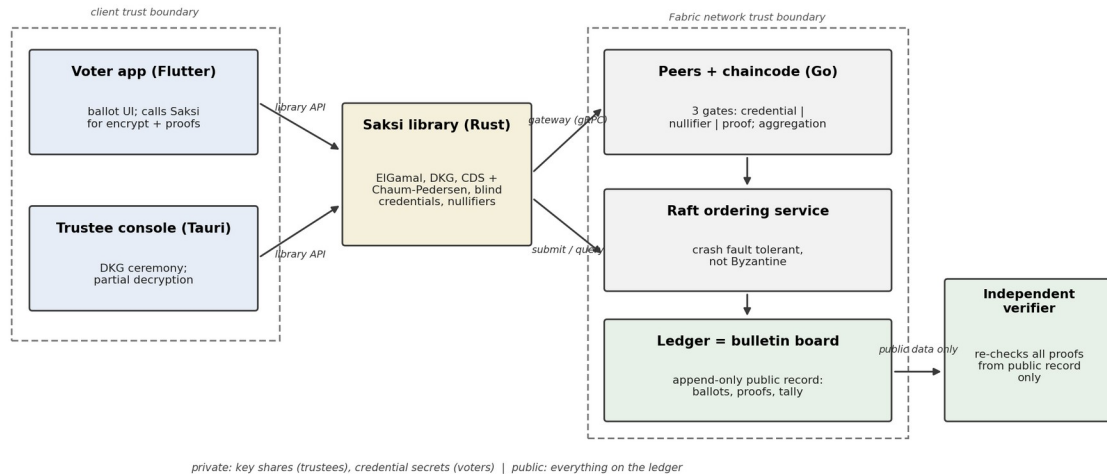


Figure 3.4. System architecture of BalotaChain: client applications, the Saksi library boundary, the Fabric network with its three chaincode gates and Raft ordering, the public ledger, the verifier path, and the trust boundaries between them.

BalotaChain comprises five components. The cryptographic core provides the primitives described in the previous section. The Hyperledger Fabric chaincode, written in Go, operates the permissioned bulletin board: it verifies each submitted ballot's credential, nullifier, and validity proof, commits valid ballots to the ledger, performs homomorphic aggregation, and coordinates the threshold decryption ceremony. The voter application, built with Flutter, provides the interface through which a voter authenticates, selects a candidate, and submits an encrypted ballot, and through which the voter can later verify that the ballot was recorded. The trustee application, built with Tauri, supports the key generation and decryption ceremonies performed by trustees. The open-source verifier allows any independent party to re-check every cryptographic operation recorded on the bulletin board. The features and their purposes are summarized in Table 3.3.

Table 3.3. System components, features, and purpose.

Component	Feature / Function	Purpose
Saksi core (Rust)	Encryption, DKG, proofs, decryption	Reusable cryptographic framework
Fabric chaincode (Go)	Ballot verification, aggregation, decryption coordination	Operates the permissioned bulletin board
Voter app (Flutter)	Authentication, ballot submission, verification	Voter-facing interface, all platforms
Trustee app (Tauri)	Key generation, partial decryption	Trustee ceremonies
Verifier (open-source)	Re-checks all cryptographic steps	Independent public verification

Software Tools

The tools used to develop, build, and evaluate the system are summarized in Table 3.4, grouped by role. The stack is composed almost entirely of free and open-source software; the indicated prices are estimates for verification and may be edited once confirmed.

Table 3.4. Software tools and estimated cost.

Name	Purpose	Price	Qty	Total
Rust toolchain	Cryptographic core development	Free	1	0
Go	Hyperledger Fabric chaincode	Free	1	0
Flutter / Dart	Voter application (all platforms)	Free	1	0
Tauri	Trustee desktop application	Free	1	0
Hyperledger Fabric	Permissioned blockchain substrate	Free	1	0
Hyperledger Caliper	Blockchain benchmarking	Free	1	0
Docker	Containerized Fabric network	Free	1	0
Git / GitHub	Version control (two repositories)	Free	1	0
VS Code	Primary editor / IDE	Free	1	0
Zotero	Reference management (IEEE)	Free	1	0

Grand Total (estimated): 0 (all listed tools are free or open-source)

Development and evaluation run on hardware the researchers already own: an Apple M4 machine for the client-side cryptographic workload, and a separate desktop that hosts the Hyperledger Fabric network. No new hardware needs to be bought.

Testing and Evaluation

The evaluation strategy measures four properties. Each one maps to a research question and produces a defined kind of result. Because the study generates its own synthetic input, the ground-truth vote counts are known, so the correctness of the system's output can be measured directly. Evaluation runs along two independent axes. The first is election scale, the number of voters, at one thousand, ten thousand, fifty thousand, four hundred eighty-three thousand, and one million, culminating in capstone tiers at 1,921,917 and 3,524,078, tiers anchored to Mindanao electoral units as defined in Data Collection. The second is ballot complexity, the number of positions per ballot, distinguishing a single-position ballot from a multi-position ballot. These two axes serve different purposes. The single-position configuration provides a controlled workload that matches the structure evaluated by the baseline system of Galal, El Reheem, and Guirguis [18], which used a single race of four candidates, so that the performance comparison required by the research questions reflects a difference in cryptography rather than a difference in workload. The multi-position configuration characterizes the system under realistic Philippine ballot conditions, in which a voter selects candidates for several positions on one ballot, and is reported on its own rather than against the baseline. Multiple independent runs are performed per configuration to support the consistency of the measurements. Table 3.5 summarizes the evaluation matrix.

Table 3.5. Evaluation matrix across the two axes.

Scale (voters), real-world analog	Single-position (baseline comparison)	Multi-position (PH-realistic)
1,000, clustered precinct (Res. 10965 cap)	Evaluated	Evaluated
10,000, barangay cluster	Evaluated	Evaluated
50,000, municipality	Evaluated (direct comparison with the 50,000-voter measurement of [18])	Evaluated
483,000, Zamboanga City (483,058 registered voters, 2025)	Evaluated	Evaluated
1,000,000, provincial-scale	Evaluated (compared with the beyond-	Stretch goal

Scale (voters), real-world analog	Single-position (baseline comparison)	Multi-position (PH-realistic)
stress test	one-million projection of [18])	
1,921,917, three Zamboanga Peninsula provinces (capstone)	Capstone attempt (committed)	Capstone attempt (committed)
3,524,078, full ZAMBASULTA (capstone)	Capstone attempt (committed)	Capstone attempt (committed)

Table 3.6 aligns each research question with the objective it serves, the system function that realises it, the scenario under which it is exercised, the metric recorded, the result expected, and the evidence retained. No metric is reported in this study that does not appear in this matrix, and no research question is left without a corresponding measurement.

Table 3.6. Alignment of research questions, objectives, system functions, scenarios, metrics, and evidence.

RQ	Objective	System function	Test scenario	Metric	Expected result	Evidence
1 (a)	Establish that the announced tally is correct	Homomorphic aggregation and three-of-five threshold decryption	T1	$E = \sum T_i - G_i $	$E = 0$ at every scale tier	Ground-truth comparison record
1 (b, c)	Establish that invalid and duplicate ballots are rejected	Chaincode proof-verification and nullifier-uniqueness gates	T4, T5	Rejection rate	All invalid and duplicate submissions rejected	Chaincode rejection log
1 (d, e, f)	Establish completeness and independent verifiability	Independent verifier operating on the public election record	T1, T7	Ballots counted; proofs verified	All accepted ballots counted; all proofs verify	Verifier output and ledger extract
2	Demonstrate ballot secrecy and voter-ballot unlinkability	Blind-signed credentials, nullifiers, and encrypted ballots	T5	Adversary linkage success rate	No better than the anonymity-set baseline	Privacy experiment record
2	Demonstrate resistance to the defined attack scenarios	Three chaincode gates and the trustee threshold	T3, T4, T5, T6	Outcome per attack scenario	Resistance demonstrated under the stated threat model	Attack simulation log
3	Characterise	Ballot	T1, T2,	Latency p50,	Reported per	Caliper performance

RQ	Objective	System function	Test scenario	Metric	Expected result	Evidence
	per-ballot latency	submission and endorsement path	T8	p95, p99	scale tier	report
3	Characterise throughput and scaling behaviour	Fabric ordering and commit path	T2, T8	Transactions per second	Reported per tier, with the scaling limit identified	Caliper performance report
3	Characterise resource use and reliability	Peer, orderer, and client resource consumption	T2, T3, T8	CPU, memory, disk, network; success rate	Reported per scale tier	Resource monitor and error logs

Testing proceeds in three tiers before any experiment is run. Unit tests cover the cryptographic primitives: encryption round-trips, proof soundness and completeness on valid and invalid statements, nullifier determinism and uniqueness, key-share generation and reconstruction at and below the threshold, and credential issuance and verification. Integration tests cover the interfaces: application to library, library to chaincode through the Fabric gateway, the three chaincode gates against the ledger, and the verifier against a populated bulletin board. A system test then executes the complete election lifecycle at the one-thousand-voter tier. All three tiers must pass before the experiments of Table 3.7 are executed.

Experimental Design

To make the evaluation reproducible, each property is operationalized as a defined experiment with an objective, a procedure, controlled variables, a primary metric, and a pass criterion. Metric definitions follow the Hyperledger Performance and Scale Working Group white paper [23], as operationalized by the Hyperledger Caliper benchmarking tool: send rate, throughput measured as committed transactions per second across the network, and transaction latency reported as percentiles rather than averages, together with success rate and resource consumption. Table 3.7 defines the experiments, and Table 3.8 defines the testing scenarios under which they run. One limitation is stated explicitly: the

Raft ordering service of Hyperledger Fabric provides crash fault tolerance, not Byzantine fault tolerance, so Byzantine behavior is addressed at the protocol layer, through proof verification and the trustee threshold, rather than at the ordering layer.

Table 3.7. Experiment definitions for the evaluation.

Experiment	Objective	Procedure and variables	Primary metric and pass criterion
Accuracy	Confirm the decrypted tally equals the seeded ground truth	Run the full pipeline at every scale tier; compare the decrypted tally with the ground-truth counts	Tally error E; pass when $E = 0$ at every tier
Integrity	Confirm invalid ballots cannot enter the count	Execute the adversarial submissions of scenarios T4 and T5	Rejection rate of invalid submissions; pass at 100 percent rejection, verifiable from the bulletin board
Reliability	Confirm failure-free repeated operation and recovery	Repeat each configuration at least three times; restart a node under load (scenario T3)	Commit success rate and ballot loss after recovery; pass at 100 percent success and zero ballot loss
Response time	Characterize per-ballot latency	Measure submission-to-commit latency under normal (T1) and peak (T8) load	Latency percentiles p50, p95, p99 per tier; characterization, no fixed threshold
Scalability	Characterize behavior as scale grows	Run the identical workload across all five tiers	Throughput and latency as functions of scale; degradation documented per tier
Throughput	Measure sustained committed transactions per second	Caliper-driven load at increasing send rates until saturation	Committed TPS at saturation per tier, compared with prior evaluations [18], [26]
Performance log analysis	Verify reported figures from system records	Post-run analysis of Caliper reports, peer and orderer logs, and chaincode events	Consistency of logs with reported metrics; no unexplained discrepancies

Table 3.8. Testing scenarios for the evaluation.

ID	Scenario	Preconditions	Test input	Action or attack	Expected behavior	Actual result and verdict	Evidence or log reference	Related security property
T1	Normal voting operations	Network healthy; registry seeded; keys generated; unit and integration tiers passed	Valid ballots at a moderate send rate	Steady submission through the complete lifecycle, registration to tally	All valid ballots accepted; E = 0; latency recorded per the Table 3.7 percentiles	Recorded at execution	Caliper report; tally record	Integrity; accuracy
T2	High-volume voter transactions	T1 passed at the same tier	Valid ballots at a high sustained send rate	Sustained load approaching system saturation	Throughput plateau documented; no accepted ballot lost	Recorded at execution	Caliper report; resource logs	Availability; performance
T3	Network interruption and recovery	A run in progress at the tier under test	Valid ballots continuing to arrive	A peer or orderer node is stopped mid-run and restarted	Ledger consistent after recovery; accepted ballots preserved; run completes or the failure is documented	Recorded at execution	Node logs; ledger extract	Reliability
T4	Attempted vote or election return tampering	Baseline election seeded; gates active	Malformed ballots; altered ciphertexts; manipulated manifests	Submission through the gates; attempted modification of committed records	All tampered inputs rejected; committed records immutable; verifier detects any modification	Recorded at execution	Chaincode rejection log; verifier output	Integrity
T5	Unauthorized access attempts	Registry fixed; credential set issued	Invalid or expired credentials; reused nullifiers; duplicate votes	Submission attempts outside an authorized identity	One hundred percent rejection, each logged with its violated gate	Recorded at execution	Chaincode rejection log	Eligibility; uniqueness
T6	Trustee validation and approval	Encrypted aggregate ready; five key shares held	Partial decryptions, including one corrupted share and one sub-threshold set	Decryption attempted with two shares, then three, including one invalid proof	Sub-threshold attempt fails; invalid proof detected; three valid shares decrypt and sign the tally	Recorded at execution	Trustee console log; decryption proofs	Ballot secrecy; threshold integrity
T7	Concurrent voting and result transmission	Multiple clients connected across positions	Parallel valid ballots for several positions	Concurrent submissions while interim aggregates are read	Per-position nullifier enforcement holds; every accepted ballot counted exactly once	Recorded at execution	Chaincode log; per-position tallies	Uniqueness; accuracy
T8	System performance under peak election	Largest tier configured; T2 completed	Valid ballots in a worst-case arrival burst sized from the tier's real-	Peak-load run at the tier ceiling	Run completes, or the bottleneck is documented as a scaling-limit finding	Recorded at execution	Caliper report; resource logs	Availability; scalability

ID	Scenario	Preconditions	Test input	Action or attack	Expected behavior	Actual result and verdict	Evidence or log reference	Related security property
	conditions		world analog					

Scenarios T1 through T8 correspond to the operational and adversarial conditions under which the experiments of Table 3.7 are executed; the mapping of each scenario to its research question and metric is given in Table 3.6. Each scenario carries its full test documentation: preconditions, test input, the action or attack performed, the expected behavior, the actual result and verdict, the evidence or log reference, and the security property it exercises. The actual-result and evidence fields are completed during the implementation phase, and every completed row is archived with the test logs in the project repositories. The adversary classes defined for the security evaluation map onto scenarios T4, T5, and T6.

Accuracy (demonstrating integrity)

Accuracy is evaluated quantitatively by comparing the system's decrypted aggregate tally against the known ground-truth counts seeded during data generation. For a correct system the two must match exactly. Letting T_i denote the decrypted count for candidate i and G_i the seeded ground-truth count, accuracy is confirmed when the tally error E is zero across all candidates:

In addition, correctness of the underlying cryptographic operations is validated against published test vectors from the ElectionGuard reference toolkit [14], so that accuracy is established against an external standard and not only against the study's own data. Cryptographic correctness is also subject to verification of every zero-knowledge proof recorded on the bulletin board. Two verification experiments make this operational. In the first, the independent verifier runs on a separate machine, is given only the public election record, and must reproduce the announced tally while accepting every proof. In the second, one public record is deliberately modified and the verifier must detect the inconsistency; detection demonstrates software independence in the sense of

Rivest [28], since an undetected change in the software or its data could not cause an undetectable change in the outcome.

Privacy

Privacy is evaluated structurally and adversarially rather than by a single numeric score, in keeping with the way privacy is treated in the cryptographic voting literature [8]. Three checks are performed. First, an unlinkability test plays the role of an adversary who holds the complete public bulletin board and tries to tie any specific voter to any specific ballot. Privacy holds if no such link can be made. Formally, the simulated adversary is given the complete public record, namely the encrypted ballots with their validity proofs, the nullifiers, the aggregate, the partial decryptions with their proofs, and the registration list of eligible voters, and must output a voter-ballot pairing. Successful linkage is defined as identifying the ballot of any targeted voter with probability better than random selection over the anonymity set of all voters at that tier, following the unlinkability terminology of Pfitzmann and Hansen [27]. Unlinkability is therefore stated as a testable hypothesis: the adversary's success rate does not exceed the reciprocal of the anonymity-set size. Second, a ballot-secrecy check confirms that no individual ballot is ever decrypted during the protocol, only the aggregate. Third, a threshold check confirms that any attempt to decrypt with fewer than the three required trustee shares fail. Each check is categorical: the property either holds or it does not.

Security

Security is checked at two levels. The protocol level tests whether the cryptographic design demonstrates resistance to the defined attacks. The implementation level tests whether the code that realizes the design is free of common vulnerabilities. A protocol can be cryptographically sound and still be exploitable through an implementation flaw, so both levels are checked.

At the protocol level, concrete attacks are written as automated, deterministic test cases, one for each defined adversary class: a malicious voter who tries to double-vote or submit a malformed ballot, a sub-threshold set of malicious trustees who try to corrupt decryption, a malicious administrator who tries to tamper with the manifest, a malicious bulletin-board node that tries to drop or reorder ballots, and a network adversary who tries interception or replay. The negative test catalog additionally exercises, as individual cases: a malformed ciphertext, an invalid or substituted validity proof, an invalid or expired credential, a reused nullifier, a duplicate vote for the same position, an altered transaction, an altered manifest, a decryption attempt with fewer than three trustee shares, an incorrect trustee decryption proof, an unauthorized identity, and corrupted ledger data; each case must be rejected by the specific gate or verification step that guards it. A rejected attack demonstrates resistance to that defined scenario under the stated threat model; it is not read as a general claim that the system is secure. Each attack is encoded as a test assertion that states the expected secure outcome, so the result is repeatable rather than the product of ad hoc manual probing. For each class, the system either rejects the attack or contains its effect within the stated trust model, and the outcome is recorded as a pass or fail for that class. The formal notions of verifiability against which these outcomes are interpreted follow the systematization of Cortier and colleagues [9].

At the implementation level, the source code of the framework and the application is subjected to vulnerability testing independent of the protocol design. Input validation testing supplies malformed, boundary, and oversized inputs at every external entry point, including ballot submission, the application interfaces, and the chaincode arguments, to confirm that invalid input is safely rejected rather than mishandled. Static application security testing is applied using the standard tooling for the stack, namely the Rust analyzers for the cryptographic core and the Go analyzers for the chaincode, to detect unsafe patterns before execution. Dependency and supply-chain scanning checks all third-party libraries against published vulnerability advisory databases so that no

component with a known vulnerability is shipped. The choice of Rust for the cryptographic core additionally removes, by language design, the classes of memory-safety vulnerabilities that commonly affect systems software. The results of implementation-level testing are recorded as the issues found and their resolution, complementing the categorical protocol-level results.

Threat Simulation and Mapping to Philippine Electoral Law

To situate the security evaluation within the Philippine legal context, each simulated adversary class is mapped to the conduct that the relevant statutes treat as an offense. The mapping draws on the Omnibus Election Code (Batas Pambansa Blg. 881), the automated election law (Republic Act No. 8436 as amended by Republic Act No. 9369), and the Data Privacy Act of 2012 (Republic Act No. 10173). Because the study uses synthetic data and processes no real voter information, the Data Privacy Act entries denote the conduct that the corresponding privacy property would prevent in an actual deployment, rather than an offense arising from the simulation itself. Table 3.10 presents the mapping.

Table 3.9 formalizes the threat model: for each adversary class it states the capability assumed, the attack attempted, the property targeted, the mechanism that defends it, and the evidence expected when the defense holds. Every attack scenario in Table 3.8 traces to a row of this matrix.

Table 3.9. Threat model matrix of adversary classes, capabilities, attacks, defenses, and expected evidence.

Adversary	Capability	Attack	Target property	Defense mechanism	Expected evidence
Malicious voter	Valid credential; controls own submissions	Double vote; malformed or invalid ballot	Integrity	Nullifier-uniqueness gate; CDS validity proof gate	Rejection logged by chaincode
Compromised client	Controls one voter's device and inputs	Substituted proof; altered ciphertext; credential misuse	Integrity; privacy	Proof verification gate; credential gate; client holds no key material	Rejection logged; no key exposure

Adversary	Capability	Attack	Target property	Defense mechanism	Expected evidence
Malicious trustees (up to two)	Hold key shares below the threshold	Corrupt or withhold partial decryptions; attempt sub-threshold decryption	Integrity; secrecy	3-of-5 threshold; Chaum-Pedersen proof per partial decryption	Invalid proof detected; decryption impossible below threshold
Network adversary	Observes and replays traffic	Interception; replay of ballots	Integrity; privacy	TLS transport; nullifier uniqueness; ciphertexts reveal nothing	Replay rejected; no plaintext observable
Ledger administrator	Operates peers or ordering nodes	Drop, reorder, or tamper with records	Integrity; verifiability	Append-only replicated ledger; independent verifier over the public record	Tampering detected by the verifier
External attacker	No credentials; network access only	Unauthorized submission; forged identity	Integrity	Credential gate; membership service	Rejection logged by chaincode

Table 3.10. Mapping of simulated adversary classes to Philippine electoral and data privacy law.

Simulated adversary (threat)	Property targeted	Corresponding Philippine legal provision
Malicious voter: double voting	Integrity (one vote per position)	Omnibus Election Code (BP 881), Sec. 261, election offenses (illegal voting); penalties under Sec. 264
Malicious voter: malformed ballot	Ballot validity	BP 881, Sec. 261(j), falsification of election documents; RA 9369, Sec. 35, automated election system integrity
Sub-threshold trustees: corrupt decryption	Integrity of the count	RA 9369 (amending RA 8436), Sec. 35(a), unauthorized tampering with the automated election system
Malicious administrator: manipulate manifest	Configuration integrity	BP 881, Sec. 261(j), falsification; RA 9369, Sec. 35(a), tampering with the automated election system
Malicious bulletin-board node: drop or reorder ballots	Append-only record integrity	RA 9369, Sec. 35(a), tampering with, damaging, or destroying election records used in the system
Network adversary: interception or replay	Confidentiality and integrity in transit	RA 9369, Sec. 35, unauthorized access to the automated election system; RA 10173, Sec. 29, unauthorized access or intentional breach
Privacy adversary: voter-ballot linkage or ballot secrecy	Privacy (unlinkability, secrecy)	RA 10173, Sec. 25, unauthorized processing; Sec. 28, processing for unauthorized purposes; Sec. 29, unauthorized access or intentional breach

Penalties for the cited election offenses are imposed under Sec. 264 of BP 881; automated election system offenses under Sec. 35 of RA 9369; and data privacy offenses under Secs. 25 to 29 of RA 10173. These

statutory references provide context for the simulated threat scenarios only; as the researchers are computer science students, no claim of legal validity, compliance, or certification is made or implied.

Election Return Verification and Approval

In the Philippine automated election system, the electronically transmitted and digitally signed election return is the official basis for the canvassing of votes (Republic Act No. 8436, Sec. 22, as amended by Republic Act No. 9369, Sec. 19), authenticated through the procedures of the Electronic Commerce Act (Republic Act No. 8792) pursuant to Sec. 30, with tampering penalized under Sec. 35. The Supreme Court has affirmed that machine-generated digital signatures satisfy this requirement and that the choice of signing mechanism lies within the discretion of the Commission on Elections (National Press Club of the Philippines v. COMELEC, G.R. No. 259354, June 13, 2023).

The proposed system strengthens this existing mechanism rather than replacing it. The final decrypted tally, the system's analogue of an election return, is approved through a multi-party ceremony: each participating trustee digitally signs the tally it helped decrypt, alongside its Chaum-Pedersen proof of honest partial decryption, and the tally is considered approved only when at least three of the five trustees have signed. The signed tally, the decryption proofs, and the trustee signatures are then committed to the bulletin board through the platform's consensus, making the approval publicly verifiable and the record immutable. This converts the single-machine digital signature of current practice into a threshold, publicly verifiable, consensus-anchored approval, while remaining compatible with the statutory framework.

Audit Trails and Logging

The bulletin board is the primary audit trail of the system: every accepted ballot with its proofs and nullifier, the encrypted aggregate, each partial decryption with its Chaum-Pedersen proof, and the signed final tally are recorded append-only on the ledger. In addition, the system retains operational logs off the chain: submission attempts and their gate outcomes, trustee ceremony actions,

node-level events, and the Caliper benchmark reports used for performance log analysis. Post-election verification proceeds from these records alone: the open-source verifier recomputes the aggregate and re-checks every proof from the bulletin board, without access to any private component. Concretely, the verifier checks, from the public record alone: (1) the validity of every recorded transaction; (2) the validity of the credential attached to each ballot; (3) the uniqueness of every nullifier; (4) the well-formedness of every ciphertext; (5) the disjunctive validity proof of every ballot; (6) the correctness of the homomorphic aggregate against the accepted ballots; (7) each trustee's Chaum-Pedersen decryption proof; (8) that at least three of the five trustees contributed; (9) that the announced tally equals the decrypted aggregate; and (10) the append-only consistency of the ledger itself.

Table 3.11 classifies every stored item by its location, visibility, purpose, and retention. No published field contains a voter identity: nullifiers and credentials are one-way values, ciphertexts are semantically secure, and the registry that could relate identities to anything at all is synthetic, off-chain, and private, so the published record cannot create a voter-ballot linkage.

Table 3.11. Data classification of stored items in the audit trail.

Item	Location	Visibility	Purpose	Retention
Encrypted ballot	On-chain	Public	Vote content under encryption; input to the aggregate	Permanent
Ballot validity proof	On-chain	Public	Shows the ballot encodes one valid selection	Permanent
Nullifier	On-chain	Public	One-way marker preventing double voting per position	Permanent
Credential token	On-chain	Public	Blind-signed proof of eligibility; unlinkable to identity	Permanent
Partial decryptions and proofs	On-chain	Public	Trustee contributions with correctness proofs	Permanent
Signed final tally	On-chain	Public	Officially approved result with trustee signatures	Permanent
Election manifest	On-chain	Public	Positions, candidates, selection limits	Permanent
Operational logs	Off-chain	Private	Gate outcomes, ceremonies, node	Research period

Item	Location	Visibility	Purpose	Retention
			events, benchmark reports	
Synthetic voter registry	Off-chain	Private	Generated eligibility list; source of ground truth	Research period

This design follows the auditability principle of the Voluntary Voting System Guidelines 2.0, under which end-to-end verifiable architectures are an accepted path to evidence-based elections [25]. The distinction the guidelines draw is preserved here: the ledger provides auditability, and the verification procedure provides the evidence. An immutable record alone does not establish a correct outcome; it is the audit procedure operating over that record that does.

Performance

Performance is evaluated quantitatively by benchmarking the system across both axes and recording per-ballot latency, transaction throughput, threshold-decryption time, and resource consumption. For the single-position configuration, results are compared against the published baseline of a voting system on the same Hyperledger Fabric platform [18], which reported approximately 1.07 seconds per vote and a throughput near 4.66 transactions per second at fifty thousand voters; as the baseline ran on different hardware, this is a cross-study comparison, not a controlled experiment. For the multi-position configuration, performance is reported on its own as a characterization of behavior under realistic Philippine ballot complexity, since no published baseline known to the researchers evaluated multi-position ballots on the same platform. Benchmarking uses repeated runs to support the stability of the reported figures. Benchmarks are executed as batch jobs on the available hardware; the tiered scales ensure that meaningful performance data are obtained even if the largest configuration, multi-position at the highest scale, proves resource-bound. The data-recording forms used to log measurements are provided in Appendix C.

Two published reference points bracket the range within which the proposed system is expected to fall. Benchmarks of Hyperledger Fabric 2.5 on

commodity server hardware report peak throughput near three thousand transactions per second with a world state of one million assets [24], while the closest prior evaluation of an ElGamal homomorphic tally on Fabric reports peak committed throughput near forty transactions per second [26]; cryptographic verification in chaincode is substantially heavier than fixed-asset benchmarks, so the proposed system is expected to sit between these bounds. Latency is reported as percentiles rather than averages, per the Experimental Design. Each configuration is executed in at least ten measured repetitions after two discarded warm-up runs; failed runs are excluded from throughput and reported separately as a failure rate. In addition to end-to-end figures, the proof-generation, proof-verification, aggregation, and threshold-decryption stages are timed separately, and every metric is reported as minimum, median, mean, ninety-fifth and ninety-ninth percentile, and standard deviation, together with processor, memory, disk, and network utilization of peers, orderers, and clients. The complete benchmarking environment is recorded in Table 3.12 and published with the results. For the provincial-scale tier, sustained throughput below approximately twenty-eight ballots per second, the arrival rate implied by one million voters over a ten-hour voting window, is reported as a scaling limit of the evaluated configuration rather than a failure of the design. The same rule applies at the capstone tiers, where the ten-hour window implies approximately fifty-four ballots per second at 1,921,917 voters and approximately ninety-eight at 3,524,078. At the two capstone tiers, the repetition count is three measured runs after one warm-up rather than ten after two, a standard accommodation for long-running benchmarks; every other element of the protocol is unchanged.

Table 3.12. Benchmarking environment specification.

Item	Specification
Processor	AMD Ryzen 7 5700G, 8 cores, 16 threads
Memory	32 GB DDR4
Storage	NVMe solid-state drive; LevelDB state database with the history database off; approximately 30 to 50 GB of ledger per million single-position ballots, with 150 GB provisioned for the full-ZAMBASULTA multi-position run
Operating system and	Recorded at execution and published with the results

Item	Specification
container runtime	
Blockchain platform	Hyperledger Fabric 2.5, Raft ordering service
Network topology	Peers, orderers, channel, and block configuration recorded at execution
Toolchain versions	Go, Rust, Flutter, Tauri, and Caliper versions recorded at execution
Container resource limits	Recorded at execution and published with the results

Ethical Considerations

Because the study uses synthetic data generated by the researchers and does not collect information from human participants, the principal privacy risks associated with electoral data are avoided by design. No real voter identities, ballots, or personal information are gathered, stored, or processed at any stage, so there is no personally identifiable information to protect and no requirement for human-subject recruitment or consent. This is a deliberate methodological choice that both strengthens validity and minimizes ethical exposure. Five commitments govern the security experiments: no real voter data are collected at any stage; no actual election is conducted; no real election credentials are issued or used; all simulated attacks are executed exclusively within the researchers' controlled experimental environment; and no external or production election infrastructure is targeted or probed.

Where the work touches established systems, the study relies only on publicly available, openly licensed software and published reference materials, and it respects the licenses of all open-source components used. The cryptographic framework and the verifier are released as open-source software, which supports transparency and allows independent scrutiny of the methods and results. The study does not deploy the system in any real election and makes no claim about the conduct of any actual electoral process; its outputs are characterization data intended to inform future research and decision-making. Should any later phase of the research involve real participants or real electoral data, appropriate institutional review and data-protection compliance, including

adherence to the Philippine Data Privacy Act, would be sought before such work proceeds.

References

- [1] M. V. Vladucu, Z. Dong, J. Medina, and R. Rojas-Cessa, "E-Voting Meets Blockchain: A Survey," *IEEE Access*, vol. 11, pp. 23293–23308, 2023, doi: 10.1109/ACCESS.2023.3253682.
- [2] U. Jafar, M. J. Ab Aziz, Z. Shukur, and H. A. Hussain, "A Systematic Literature Review and Meta-Analysis on Scalable Blockchain-Based Electronic Voting Systems," *Sensors*, vol. 22, no. 19, art. 7585, 2022, doi: 10.3390/s22197585.
- [3] V. C. Reyes Jr., "The Impact of Automation on Elections: Case Study of the May 2010 Philippine Presidential Contests," *Journal of Developing Societies*, vol. 29, no. 3, pp. 259–285, 2013, doi: 10.1177/0169796X13494276.
- [4] T. ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985, doi: 10.1109/TIT.1985.1057074.
- [5] T. P. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," in *Advances in Cryptology — CRYPTO '91*, LNCS vol. 576, Springer, 1992, pp. 129–140, doi: 10.1007/3-540-46766-1_9.
- [6] D. Chaum and T. P. Pedersen, "Wallet Databases with Observers," in *Advances in Cryptology — CRYPTO '92*, LNCS vol. 740, Springer, 1993, pp. 89–105, doi: 10.1007/3-540-48071-4_7.
- [7] R. Cramer, I. Damgård, and B. Schoenmakers, "Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols," in *Advances in Cryptology — CRYPTO '94*, LNCS vol. 839, Springer, 1994, pp. 174–187, doi: 10.1007/3-540-48658-5_19.
- [8] D. Chaum, P. Y. A. Ryan, and S. Schneider, "A Practical Voter-Verifiable Election Scheme," in *Computer Security — ESORICS 2005*, LNCS vol. 3679, Springer, 2005, pp. 118–139, doi: 10.1007/11555827_8.

- [9] B. Adida, “Helios: Web-based Open-Audit Voting,” in Proc. 17th USENIX Security Symposium, San Jose, CA, USA, 2008, pp. 335–348.
- [10] M. R. Clarkson, S. Chong, and A. C. Myers, “Civitas: Toward a Secure Voting System,” in Proc. 2008 IEEE Symposium on Security and Privacy, 2008, pp. 354–368, doi: 10.1109/SP.2008.32.
- [11] A. Juels, D. Catalano, and M. Jakobsson, “Coercion-Resistant Electronic Elections,” in Proc. 2005 ACM Workshop on Privacy in the Electronic Society (WPES ’05), 2005, pp. 61–70, doi: 10.1145/1102199.1102213.
- [12] P. Y. A. Ryan, P. B. Rønne, and V. Iovino, “Selene: Voting with Transparent Verifiability and Coercion-Mitigation,” in Financial Cryptography and Data Security 2016 Workshops, LNCS vol. 9604, Springer, 2016, pp. 176–192, doi: 10.1007/978-3-662-53357-4_12.
- [13] V. Cortier, P. Gaudry, and S. Glondou, “Belenios: A Simple Private and Verifiable Electronic Voting System,” in Foundations of Security, Protocols, and Equational Reasoning, LNCS vol. 11565, Springer, 2019, pp. 214–238, doi: 10.1007/978-3-030-19052-1_14.
- [14] J. Benaloh, M. Naehrig, O. Pereira, and D. S. Wallach, “ElectionGuard: A Cryptographic Toolkit to Enable Verifiable Elections,” in Proc. 33rd USENIX Security Symposium, Philadelphia, PA, USA, 2024, pp. 5485–5502.
- [15] P. McCorry, S. F. Shahandashti, and F. Hao, “A Smart Contract for Boardroom Voting with Maximum Voter Privacy,” in Financial Cryptography and Data Security 2017, LNCS vol. 10322, Springer, 2017, pp. 357–375, doi: 10.1007/978-3-319-70972-7_20.
- [16] J. Díaz-Santiso and P. Fraga-Lamas, “E-Voting System Using Hyperledger Fabric Blockchain and Smart Contracts,” Engineering Proceedings, vol. 7, no. 1, art. 11, 2021, doi: 10.3390/engproc2021007011.
- [17] B. M. B. Pereira, J. M. Torres, P. M. Sobral, R. S. Moreira, C. P. de A. Soares, and I. Pereira, “Blockchain-Based Electronic Voting: A Secure and Transparent Solution,” Cryptography, vol. 7, no. 2, art. 27, 2023, doi: 10.3390/cryptography7020027.
- [18] M. Galal, E. A. E. Reheem, and S. Guirguis, “Enhancing Privacy and Transparency in Electronic Voting: A Blockchain-Based Cryptographic Framework,” Journal of Cloud Computing, vol. 15, art. 39, 2026, doi: 10.1186/s13677-026-00839-z.
- [19] V. Cortier, D. Galindo, R. Küsters, J. Müller, and T. Truderung, “SoK: Verifiability Notions for E-Voting Protocols,” in Proc. 2016 IEEE Symposium on Security and Privacy, 2016, pp. 779–798, doi: 10.1109/SP.2016.52.
- [20] P. Ehin, M. Solvak, J. Willemson, and P. Vinkel, “Internet Voting in Estonia 2005–2019: Evidence from Eleven Elections,” Government Information Quarterly, vol. 39, no. 4, art. 101718, 2022, doi: 10.1016/j.giq.2022.101718.

Appendix A: Data Collection

Sample synthetic voter records (multi-position configuration)

voter_id	ballot_complexity	position	anonymous_credential	nullifier	candidate_selection
V-000001	multi	PRESIDENT	0x8f3a...9b21	0x4c1e...a7d0	CAND_P_02
V-000001	multi	VICE PRESIDENT	0x8f3a...9b21	0x91b7... e3f5	CAND_V_01
V-000001	multi	SENATOR	0x8f3a...9b21	0x2d60... c84a	CAND_S_03
V-000002	multi	PRESIDENT	0xa017... 5e44	0x7fa2... 10bd	CAND_P_01
V-000002	multi	VICE PRESIDENT	0xa017... 5e44	0xbe39... 6c22	CAND_V_02
V-000002	multi	SENATOR	0xa017... 5e44	0x05cd... 9af1	CAND_S_01
V-000003	single	PRESIDENT	0x33dd... 8b07	0xc7e8... 41a9	CAND_P_04
V-000004	single	PRESIDENT	0x6b92... f2ce	0x1a44... d3b6	CAND_P_02

Credential and nullifier values are truncated for display. Within a multi-position ballot, the three records for one voter share a single voter_id but carry independent nullifiers, one per position, so that double voting is prevented per voter per position.

Generation parameters

Parameter	Value / Description
Election scales (voters)	1,000; 10,000; 50,000; 483,000; 1,000,000; 1,921,917 and 3,524,078 (capstone)
Ballot complexity	Single-position; multi-position (three positions: President, Vice President, Senator, single-winner each)
Records per voter	One record per position (single-position: one; multi-position: three)
Cryptographic group	ristretto255 (prime-order elliptic curve group)
Candidate set per position	Fixed set of K candidates; selection drawn from a defined categorical distribution
Selection distribution	Configurable per position (uniform and skewed profiles), fixed for reproducibility
Anonymous credential	Blind-signed credential issued at simulated registration; unlinkable to voter_id
Nullifier	Single-use value derived per voter per position; prevents double voting per position
Ground truth	Per-position vote-to-candidate counts known at generation; basis for tally error $E = \sum T - G $
Randomness and seed	Deterministic pseudorandom generation with a recorded seed for reproducible populations
Data nature	Fully synthetic; no human subjects, no personally identifiable information

Sample seeded ground truth (single-position, 500-voter sample)

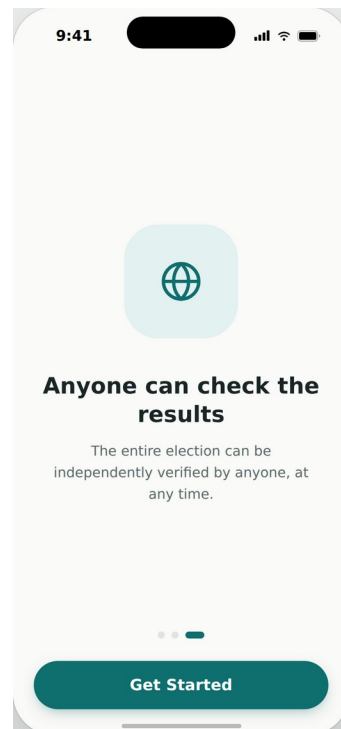
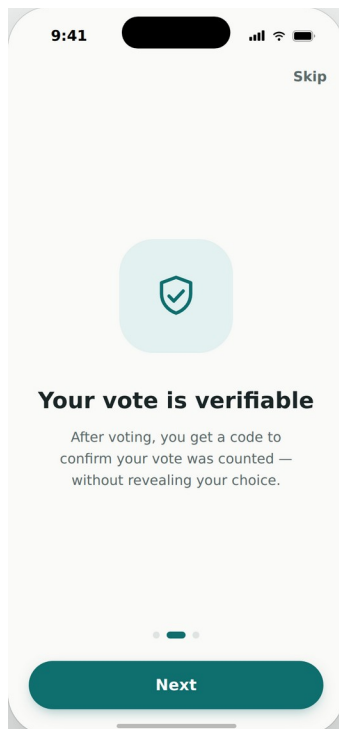
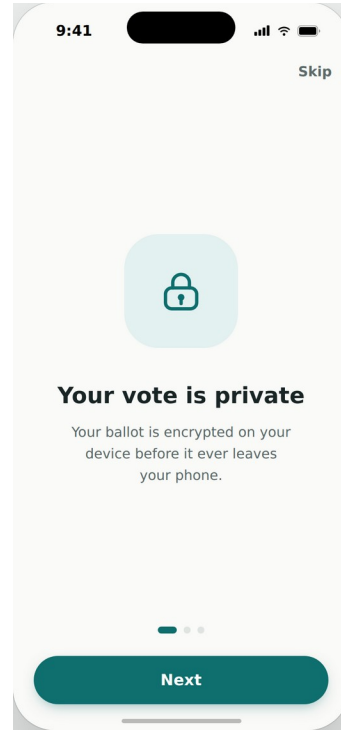
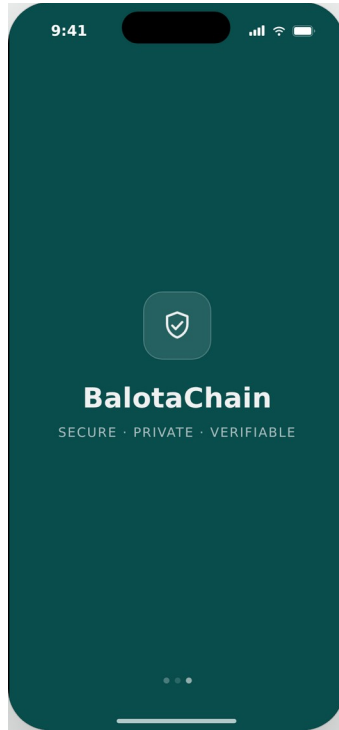
Position	Candidate	Seeded ground-truth count
PRESIDENT	CAND_P_01	142
PRESIDENT	CAND_P_02	119

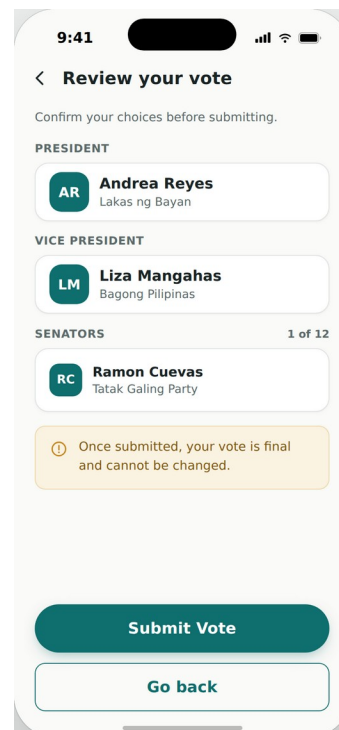
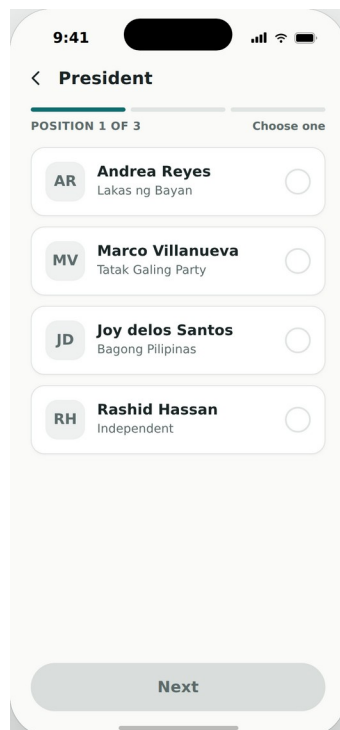
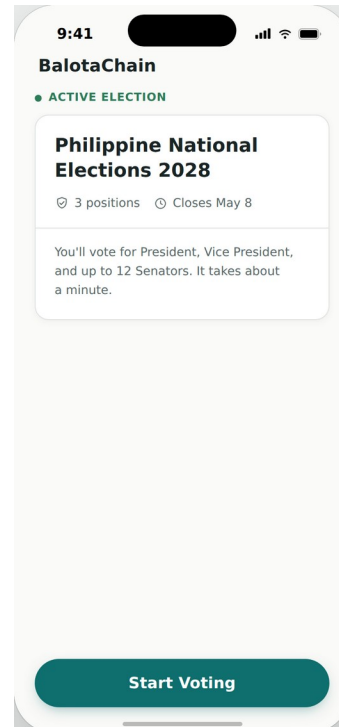
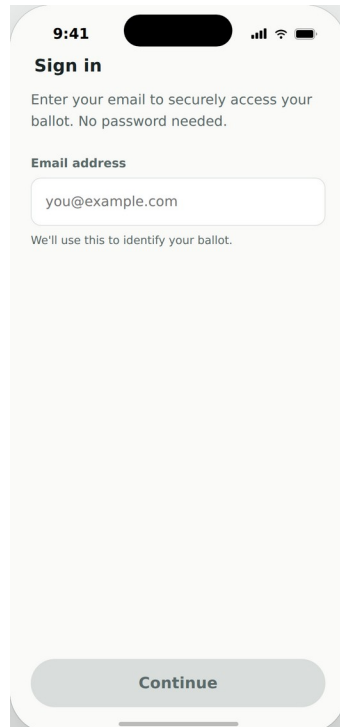
PRESIDENT	CAND_P_03	131
PRESIDENT	CAND_P_04	108
	Total	500

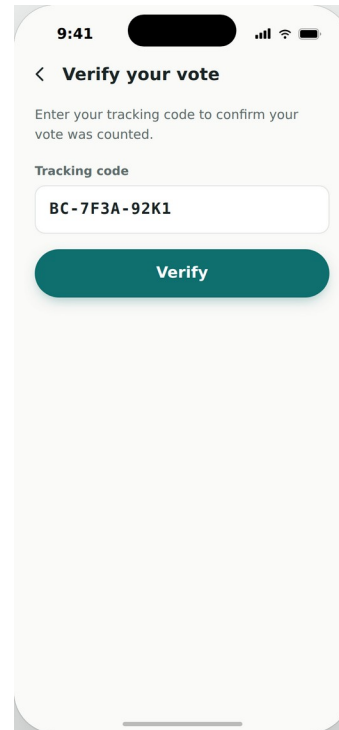
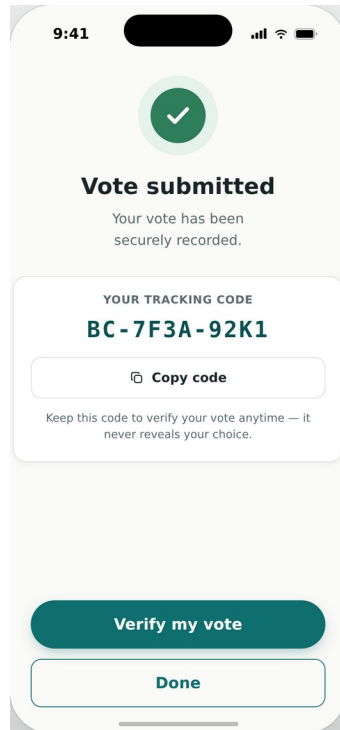
The seeded counts above define the ground truth for one single-position configuration at the 500-voter scale. After homomorphic aggregation and threshold decryption, the system's decrypted counts are compared against these values; an exact match yields a tally error of zero.

Appendix B: Software Design/Prototype

Voter Application







Public Bulletin Board



Election Closed

PUBLIC BULLETIN BOARD

Philippine National Elections 2028

Polls closed May 8, 2028 · 7:00 PM PHT · Tally published May 8, 2028 · 9:42 PM

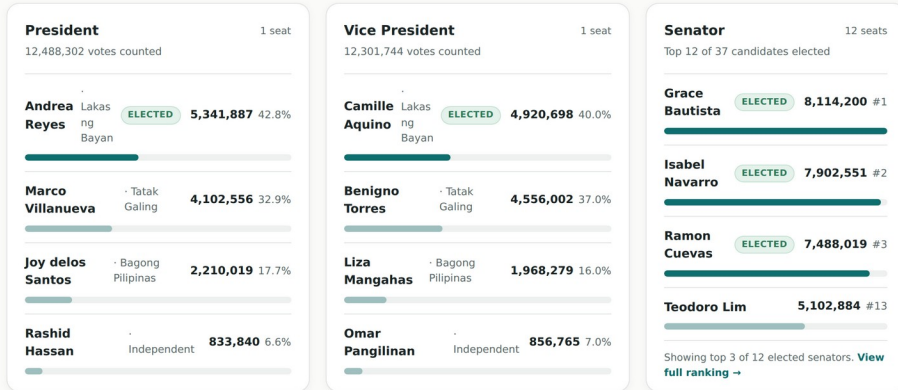


Election verified — all ballots accounted for

The complete tally has been cryptographically confirmed and independently reproduced. No ballots were added, removed, or altered.

FINAL RESULTS

3 positions · 100% of precincts reporting

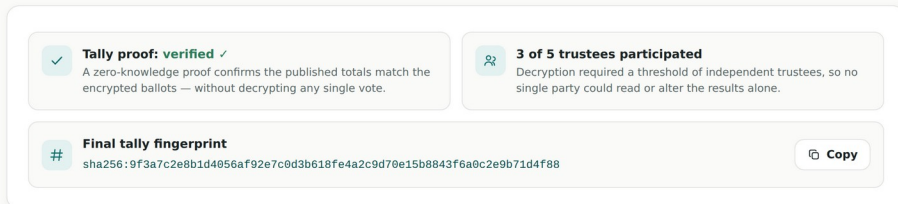


INTEGRITY SUMMARY



CRYPTOGRAPHIC VERIFICATION

Anyone can reproduce these checks with the open verifier



Verify your vote

Paste the tracking code from your receipt to confirm your ballot was included in the final tally. Confirm your ballot was included without revealing your choice.

e.g. BC-7F3A-92K1

Verify

The BalotaChain bulletin board and verifier are fully open-source. Anyone can download the encrypted ballot record and independently re-run every check on their own machine — no trust in the operator required.

[Download verification data](#)

[Open verifier](#)

Trustee Console

Secure session

RL

Roberto Lim

Trustee 2 of 5

ACTIVE CEREMONY

Decryption Ceremony

Philippine National Elections 2028 · polls closed May 8, 2028 · 7:00 PM PHT

Threshold Decryption

QUORUM 3 OF 5

3 of 5 trustees are required to decrypt the final tally.

2 of 3 submitted

1 more partial decryption needed to reach the threshold.

MS

Maria Santos

Commission on Elections

Submitted

RL

Roberto Lim

Independent Election Observer

Pending

DC

David Cruz

Parish Pastoral Council for Responsible Voting

Submitted

ER

Elena Reyes

University IT Department

Offline

AF

Ahmad Faisal

National Citizens' Movement for Free Elections

Pending

WHAT YOU MUST DO

Submit your partial decryption

Your key share is combined with the others to decrypt **only the final totals** — never any individual ballot. The threshold is met once three trustees submit.

Submit Partial Decryption

This action is irreversible and is permanently recorded to the public bulletin board.

What is being decrypted

VERIFICATION CONTEXT

Encrypted aggregate tally — fingerprint

sha256:9f3a...71d4f88

Copy

Ballots aggregated

12,611,219 verified ballots

Positions in tally

3 President · Vice President · Senator

This tally is the homomorphic aggregate of all ballots. Individual ballots are never decrypted — only the combined totals are revealed.

Ready — awaiting your share

Your identity & key share

Roberto Lim

Trustee 2 of 5 · Independent Election Observer

Key share status

Held securely

Share fingerprint

ks2:4a7f...e0c1

From ceremony

Key generation · May 1

Your private key share never leaves this device. Only a partial decryption — which reveals nothing on its own — is transmitted.

Ceremony audit log

READ-ONLY

May 1, 2028 · 09:12:40

Key generation ceremony completed — 5 of 5 trustees, threshold set to 3

May 8, 2028 · 07:00:00

Election opened — ballots accepted to bulletin board

May 8, 2028 · 19:00:00

Election closed — 12,613,540 ballots sealed

May 8, 2028 · 21:30:15

Homomorphic aggregation completed — 12,611,219 verified ballots

May 8, 2028 · 21:42:00

Decryption ceremony opened

Every action in this ceremony is signed and recorded to the public bulletin board, where anyone can independently verify it.

68

Appendix C: Evaluation Tool

This appendix presents the data-recording instruments used during evaluation. The forms are organized by the four evaluation dimensions of the study: performance and accuracy are recorded quantitatively, while privacy and security are recorded categorically as structural checks and as pass or fail outcomes against defined adversary classes. One performance and one accuracy form are completed per configuration; the privacy and security forms are completed once per evaluation campaign.

Performance measurement form

Configuration	Scale (voters)	Ballot complexity	Latency / ballot (s)	Throughput (TPS)	Threshold-decrypt (s)	Resources
SP-1K	1,000	Single-position				
SP-10K	10,000	Single-position				
SP-50K	50,000	Single-position				
SP-483K	483,000	Single-position				
SP-1M	1,000,000	Single-position				
SP-1.9M	1,921,917 (capstone)	Single-position				
SP-3.5M	3,524,078 (capstone)	Single-position				
MP-1K	1,000	Multi-position (3)				
MP-10K	10,000	Multi-position (3)				
MP-50K	50,000	Multi-position (3)				
MP-483K	483,000	Multi-position (3)				
MP-1M	1,000,000	Multi-position (3)				
MP-1.9M	1,921,917 (capstone)	Multi-position (3)				
MP-3.5M	3,524,078 (capstone)	Multi-position (3)				

Resources records peak CPU utilization and peak memory. The single-position rows are compared against the published baseline on the same Hyperledger Fabric platform; the multi-position rows are reported on their own as the Philippine-realistic characterization. The one-million-voter multi-position configuration is the provincial-scale stretch goal.

Accuracy and integrity recording form

Configuration	Position	Candidate	Decrypted count	Ground-truth count	Absolute difference

The tally error E is the sum of the absolute differences across all candidates and positions for a configuration; $E = 0$ indicates an exact tally and therefore demonstrates integrity. Decrypted counts are validated against the ElectionGuard test vectors, and cryptographic correctness is further checked by verifying every zero-knowledge proof recorded on the bulletin board.

Privacy evaluation form

Check	Description	Method	Result	Notes
Unlinkability	An adversary holding the full public bulletin board attempts to link a voter to a ballot.	Structural analysis of the bulletin board under the unlinkability adversary.		
Ballot secrecy	No individual ballot is ever decrypted; only the homomorphic aggregate is decrypted.	Inspection of the tally path; assertion that no single-ballot decryption occurs.		
Sub-threshold resistance	Decryption attempted with fewer than three of five trustee shares.	Automated test asserting that sub-threshold decryption fails.		

Security evaluation form

Level	Adversary / test	Expected outcome	Result	Notes
Protocol	Malicious voter attempts double voting or a malformed ballot.	Double vote blocked by the nullifier; malformed ballot rejected by the well-formedness proof.		
Protocol	Sub-threshold set of malicious trustees attempts to corrupt decryption.	Decryption fails; the aggregate remains secret.		
Protocol	Malicious administrator attempts to manipulate the manifest.	Manifest manipulation is detected and rejected on verification.		
Protocol	Malicious bulletin-board node attempts to drop or reorder ballots.	Omission or reordering is detected by proof and hash verification.		
Protocol	Network adversary attempts interception or replay.	End-to-end verifiability and integrity are preserved.		
Implementation	Input validation at every external entry point (ballot submission, application interfaces, chaincode arguments).	Malformed, boundary, and oversized inputs are safely rejected.		
Implementation	Static application security testing: Rust analyzers for the core, Go analyzers for the chaincode.	Unsafe patterns are detected before execution.		
Implementation	Dependency and supply-	No component with a		

on	chain scanning against published advisory databases.	known vulnerability is shipped.		
Implementation	Memory-safety vulnerability classes.	Removed by language design through use of Rust for the core.		

Appendix D: Research Budget

The software cost of the study is zero: every component, including Rust, Go, Flutter, Tauri, Hyperledger Fabric, and Hyperledger Caliper, is free and open-source, and the frameworks produced by the study are themselves released under open-source licenses. A zero software cost, however, is distinct from the total cost of conducting the research, which is estimated below and borne by the researchers.

Estimated research expenses: electricity for development and benchmarking, PHP 2,500, reflecting multi-day continuous runs at the capstone tiers; internet connectivity (prorated share over the research period), PHP 2,400; transportation for consultations and defense, PHP 1,000; printing and binding of manuscripts and forms, PHP 3,000; cloud services and hosting, PHP 0, as all benchmarking is executed locally; and depreciation of existing computing equipment prorated to the research period, PHP 4,000. The estimated total research cost is PHP 12,900. Amounts are estimates recorded for transparency and are updated to actual figures upon completion of the study.



Western Mindanao State University
College of Computing Studies
DEPARTMENT OF COMPUTER SCIENCE
Zamboanga City



RESEARCH BUDGET PROPOSAL

Research Title: BalotaChain on the Saksi Framework: End-to-End Verifiable Elections on Permissioned Blockchain Using Synthetic Election Data

Proponents: Vaniel John Cornelio, Frederick Bejerano, Idris Bakau

Supplies and Materials Expenses (Please itemize)	
1. Bond paper, ink, and printing consumables for drafts and revisions	Php1,000.00
2. Green folders and binder clips for submission	Php200.00
Sub-total	Php1,200.00
Printing and Publication Expenses	
1. Proposal Oral Defense x 3 copies at 0.00 each	Php0.00
2. Final Oral Defense x 3 copies at 0.00 each	Php0.00
3. Manuscript for hardbound x 1 copy at 500.00 each	Php500.00
4. IEEE International Presentation and Publication	Php10,000.00
Sub-total	Php10,500.00
Representation Expenses (e.g. food for meetings, etc.)	
1. Coordination meetings at cafe (food and beverages), recurring	Php4,000.00
2. Adviser consultation meetings (refreshments)	Php1,500.00
3. Development sprint working sessions (food and beverages)	Php1,500.00
4. Meals during defense preparation sessions	Php1,000.00
Sub-total	Php8,000.00
Transportation	
1. Fuel, gas, and fares for meetings, consultations, and defense	Php4,500.00
Sub-total	Php4,500.00
Subscription Expenses	
1. Not applicable (local benchmarking; no cloud subscription)	Php0.00
Sub-total	Php0.00
Other Professional Services	
1. Not applicable (synthetic data; no expert annotation)	Php0.00
Sub-total	Php0.00
Grand Total	Php24,200.00

Prepared by:

Student Name/Researchers	Signature
--------------------------	-----------



Western Mindanao State University
College of Computing Studies
DEPARTMENT OF COMPUTER SCIENCE
Zamboanga City



1. Vaniel John Cornelio	
2. Frederick Bejerano	
3. Idris Bakaun	

